

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN
MÔN THIẾT KẾ MẠCH TÍCH
HỢP SỐ

ĐỀ TÀI:

THIẾT KẾ RTL và tổng hợp FPGA thuật toán
CORDIC tính hàm căn bậc hai

Giảng viên hướng dẫn:
TS. Nguyễn Kiêm Hùng

Sinh viên thực hiện:
Bùi Hoàng Giang – 23021805
Đặng Hoài Nam – 23021869

Hà Nội, Tháng 5 Năm 2026

Mục lục

1	Giới thiệu về báo cáo	2
2	Giao diện ghép nối I/O	2
3	Thuật toán	4
4	Thiết kế mức RTL	6
4.1	Mô hình máy FSM	6
4.2	Phân tích chi tiết các Module chính và Giao tiếp (Interface) . . .	22
4.3	Module top: SQRT	22
4.4	Khối Điều khiển: Controller	22
4.5	Khối Dữ liệu: Datapath	22
4.6	Mã nguồn VHDL Khối SQRT (Top-level)	23
4.7	Mã nguồn VHDL Khối Datapath	25
4.8	Mã nguồn VHDL Khối Controller	29
4.9	Mã nguồn VHDL Khối Testbench (Mô phỏng)	35
5	Mô phỏng/thực thi và đánh giá	36
5.1	Mô phỏng trên ModelSim	37
6	Triển khai trên FPGA	37
6.1	Tổng hợp kết quả trên Vivado	38
7	Đánh giá sau Post-Implementation	42
7.1	Kết quả mô phỏng	42
7.2	Phân tích Timing	43
7.3	Tính toán tần số hoạt động tối đa	43
7.4	Kết luận	43
7.5	So sánh Functional Simulation giữa các giai đoạn	44
7.6	Report Summary	45
7.7	So sánh tài nguyên sử dụng của Post Synthesis và Port Implementation	46
8	Kết luận cả dự án	47
8.1	Những gì đã đạt được	47
8.2	Lời kết	48

1 Giới thiệu về báo cáo

Ver 1.0
29/3/2026

	Họ và tên (Full name)	Mã SV (ID)	Đóng góp (Contribution)
Thành viên 1 (Member 1)	Đặng Hoài Nam	23021869	Vẽ FSMĐ, xây dựng khối controller, datapath, mô hình hóa bằng VHDL bản final version
Thành viên 2 (Member 2)	Bùi Hoàng Giang	23021805	mô hình hóa VHDL controller, datapath và các module liên quan của version 0.2, viết báo cáo
Tên/Địa chỉ Repo trên Github hoặc Google Drive	https://github.com/hnam123/SQRT_final		

Tóm tắt (Abstract - from 5 to 10 lines)
Bài báo cáo dưới đây trình bày quá trình thực hiện bài tập lớn thiết kế RTL và tổng hợp FPGA của thuật toán cordic cho hàm căn bậc hai. Bài trình bày đã đạt được tối đa các yêu cầu của giảng viên như vẽ và mô hình hóa FSMĐ, CONTROLLER, DATAPATH, kiểm chứng bằng Modelsim và Vivado chứng minh mạch hoạt động đúng tính năng.

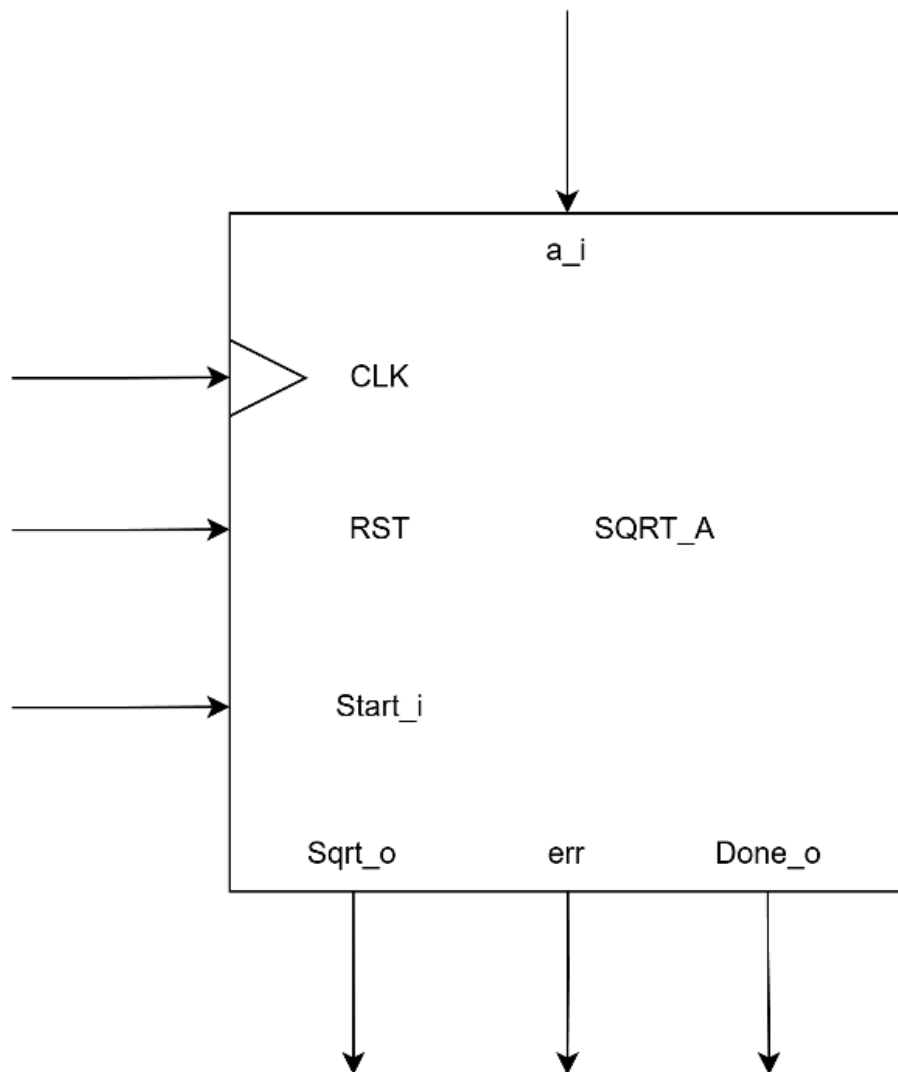
Document History

Version	Time	Revised by	Description
V0.1	29/03/2026	Nguyễn Kiêm Hùng	Original Version
V0.3	20/4/2026	Đặng Hoài Nam	Final Version
V0.2	20/04/2026	Bùi Hoàng Giang	V2.0 version

2 Giao diện ghép nối I/O

- Khối SQRTApprox có giao diện ghép nối tới CPU sao cho CPU có thể ghi giá trị đầu vào cần tính toán (giá trị a) tới khối xử lý.

- CPU kích hoạt quá trình tính toán của khối SQRTApprox bằng cách đặt tín hiệu Start = '1'.
- Sau khi quá trình tính $\sqrt{a}$ hoàn thành, khối SQRTApprox sẽ thông báo cho CPU bằng cách đặt tín hiệu Done = '1'.



Hình 1: Giao diện ghép nối I/O

Bảng 1. Mô tả các tín hiệu vào/ra

TT	Port	Direction	Width	Meaning
1	CLK	IN	1	Cấp xung clock hoạt động cho mạch
2	RST	IN	1	Reset mạch về trạng thái ban đầu
3	Start_i	IN	1	Cho phép mạch bắt đầu tính toán
4	a_i	IN	16	Giá trị đầu vào cần tính căn bậc hai
5	Done_o	OUT	1	Tín hiệu thông báo mạch đã hoàn thành tính toán
6	Err	OUT	1	Tín hiệu báo lỗi khi đầu vào không hợp lệ
7	Sqrt_o	OUT	16	Giá trị căn bậc hai của đầu vào tại ngõ ra

3 Thuật toán

Thuật toán tính căn bậc hai bằng CORDIC Thuật toán sử dụng phương pháp CORDIC hyperbolic để tính gần đúng căn bậc hai của một số thực dương ở dạng fixed-point.

Thuật toán 1 Thuật toán CORDIC tính căn bậc hai

Input: a : số thực dương dạng fixed-point

Output: $\text{sqrt_a} \approx \sqrt{a}$ và cờ lỗi error

```
1:  $N \leftarrow 32$ 
2:  $K \leftarrow 1.2075$ 
3:  $\text{error} \leftarrow 0$ 
4: if  $a = 0$  then
5:   return 0
6: end if
7: if  $a < 0$  then
8:    $\text{error} \leftarrow 1$ 
9:   return 0
10: end if
```

Bước 1: Chuẩn hóa dữ liệu đầu vào

```
11:  $\text{scale} \leftarrow 0$ 
12: while  $a < 0.5$  do
13:    $a \leftarrow a \times 4$ 
14:    $\text{scale} \leftarrow \text{scale} - 1$ 
15: end while
16: while  $a \geq 2$  do
17:    $a \leftarrow a/4$ 
18:    $\text{scale} \leftarrow \text{scale} + 1$ 
19: end while
```

Bước 2: Khởi tạo giá trị ban đầu

```
20:  $x \leftarrow a + 0.25$ 
21:  $y \leftarrow a - 0.25$ 
```

Bước 3: Thực hiện vòng lặp CORDIC

```
22: for  $i \leftarrow 1$  to  $N$  do
23:   if  $y < 0$  then
24:      $d \leftarrow 1$ 
25:   else
26:      $d \leftarrow -1$ 
27:   end if
28:    $\text{shift} \leftarrow 2^{-i}$ 
29:    $x_{\text{new}} \leftarrow x + d \times y \times \text{shift}$ 
30:    $y_{\text{new}} \leftarrow y + d \times x \times \text{shift}$ 
31:    $x \leftarrow x_{\text{new}}$ 
32:    $y \leftarrow y_{\text{new}}$ 
33: end for
```

Bước 4: Hiệu chỉnh hệ số khuếch đại

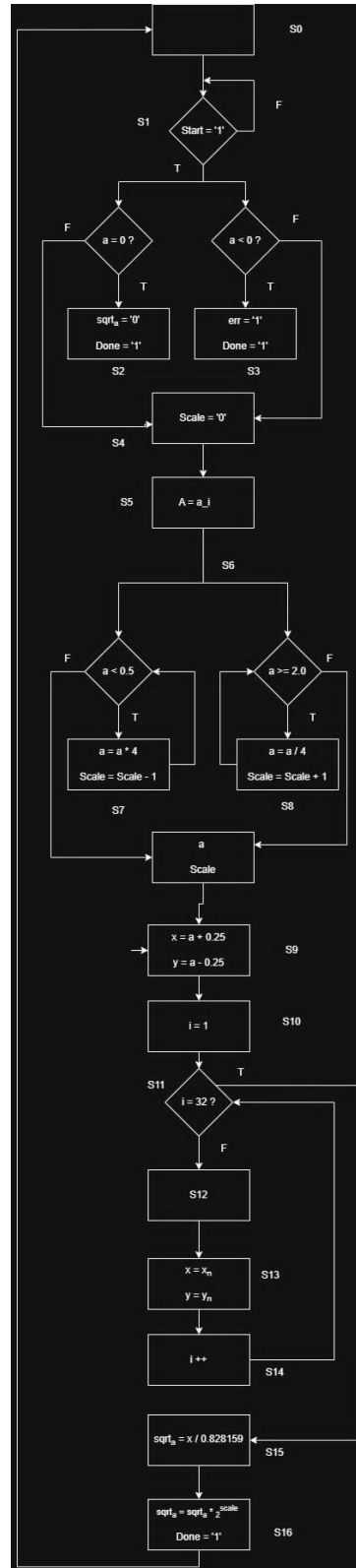
```
34:  $\text{sqrt\_a} \leftarrow \frac{x}{K}$ 
```

Bước 5: Khôi phục tỉ lệ chuẩn hóa

```
35:  $\text{sqrt\_a} \leftarrow \text{sqrt\_a} \times 2^{\text{scale}}$ 
36: return  $\text{sqrt\_a}$ 
```

4 Thiết kế mức RTL

4.1 Mô hình máy FSMD



Hình 2: Máy FSMD

Bảng định nghĩa các tín hiệu và chân giao tiếp:

Tên Tín hiệu / Chân	Phân loại	Độ rộng (Bit)	Ý nghĩa và Chức năng quản lý
Start	Input	1 bit	Tín hiệu điều khiển kích hoạt hệ thống bắt đầu tính toán ($\text{Start} = '1'$).
a	Input / Reg	Tùy chọn (Fixed-point)	Thanh ghi lưu giá trị đầu vào cần tính căn bậc hai.
Scale	Internal Reg	Số nguyên (Signed)	Thanh ghi dịch bit dữ liệu, dùng để chuẩn hóa và tái chuẩn hóa giá trị a .
x, y	Internal Reg	Tùy chọn (Fixed-point)	Cặp thanh ghi tọa độ Hyperbolic phục vụ thuật toán lặp CORDIC.
i	Internal Reg	6 bit	Bộ đếm số chu kỳ lặp (Quét từ giá trị 1 đến 32).
d	Internal Wire	2 bit (Signed)	Hướng xoay vector ('1' tương đương 1, '-1' tương đương -1).
shift	Internal Wire	Tùy chọn	Giá trị lượng tử dịch bit tại bước thứ i , có giá trị bằng 2^{-i} .
sqrt_a	Output	Tùy chọn (Fixed-point)	Kết quả cuối cùng của phép toán tính căn bậc hai.
Done	Output	1 bit	Cờ báo hiệu chu kỳ tính toán hoàn thành, dữ liệu đầu ra đã sẵn sàng.
err	Output	1 bit	Cờ báo hiệu lỗi dữ liệu đầu vào không hợp lệ (Trường hợp $a < 0$).

Mô tả hoạt động của mô hình máy FSM:

Giai đoạn 1: Trạng thái chờ và Kiểm tra điều kiện biên

- **Bước 1:** Hệ thống khởi tạo ở trạng thái chờ (Idle). Mạch liên tục kiểm tra chân Start. Nếu $\text{Start} = '0'$, hệ thống giữ nguyên trạng thái chờ. Khi $\text{Start} = '1'$, mạch bắt đầu nạp dữ liệu a vào hệ thống.
- **Bước 2: Kiểm tra song song điều kiện đặc biệt:**
 - **Trường hợp $a = 0$:** Nếu điều kiện đúng, hệ thống lập tức gán kết quả đầu ra $\text{sqrt_a} = 0$, bật cờ $\text{Done} = '1'$ và quay về trạng thái Idle (Bỏ qua toàn bộ các bước tính toán sau).
 - **Trường hợp $a < 0$:** Nếu điều kiện đúng (Phép toán căn bậc hai số thực không tồn tại), hệ thống lập tức bật cờ báo lỗi $\text{err} = '1'$, dựng cờ hoàn thành $\text{Done} = '1'$ và thoát thẳng về trạng thái Idle.
 - **Trường hợp dữ liệu hợp lệ ($a > 0$):** Hệ thống chuyển sang giai đoạn chuẩn hóa dữ liệu.

Giai đoạn 2: Vòng lặp chuẩn hóa dữ liệu động Giai đoạn này sử dụng vòng lặp để đưa giá trị a về khoảng hội tụ lý tưởng $[0.5, 2.0)$ của hàm Hyperbolic.

- **Bước 1:** Khởi tạo thanh ghi quản lý tỷ lệ $\text{Scale} = 0$.
- **Bước 2: (Vòng lặp thu hẹp/mở rộng a):**
 - **Vòng lặp $a < 0.5$:** Nếu a nhỏ hơn 0.5, mạch thực hiện phép dịch trái nhân 4 ($a = a \times 4$) và trừ thanh ghi Scale đi 1 đơn vị ($\text{Scale} = \text{Scale} - 1$), sau đó quay ngược lại kiểm tra điều kiện cho đến khi $a \geq 0.5$.
 - **Vòng lặp $a \geq 2.0$:** Nếu a lớn hơn hoặc bằng 2.0, mạch thực hiện phép dịch phải chia 4 ($a = a/4$) và cộng thanh ghi Scale lên 1 đơn vị ($\text{Scale} = \text{Scale} + 1$), sau đó lặp lại cho đến khi $a < 2.0$.

Khi a đã nằm trọn trong khoảng $[0.5, 2.0)$, giá trị cuối cùng của a và Scale được chốt.

Giai đoạn 3: Khởi tạo cấu hình ma trận CORDIC Mạch đường dữ liệu tiến hành thiết lập các giá trị hình học ban đầu cho các thanh ghi tọa độ:

- Thanh ghi x nhận giá trị: $x = a + 0.25$
- Thanh ghi y nhận giá trị: $y = a - 0.25$
- Đặt bộ đếm chỉ số vòng lặp: $i = 32$

Giai đoạn 4: Vòng lặp lõi CORDIC Hyperbolic (Chu kỳ lặp 31 bước) Hệ thống xử lý khối lặp tuần tự từ bước $i = 1$ cho đến bước $i = 32$:

- **Bước 1 (Kiểm tra điều kiện dừng $i = 32$?):**
 - Nếu **Đúng (T)** (Bộ đếm đạt đến chu kỳ thứ 32, tức đã hoàn thành xong 31 bước lặp), hệ thống phá vỡ vòng lặp, chuyển xuống khối xử lý sau lặp.
 - Nếu **Sai (F)**, hệ thống tiếp tục triển khai các phép tính toán học trong chu kỳ thứ i :
- **Bước 2 (Xác định hướng dịch chuyển và tính độ dịch):**
 - Kiểm tra dấu của thanh ghi y thông qua khối $y < 0$: Nếu đúng, hướng xoay $d = 1$, nếu sai hướng xoay $d = -1$.
 - Khối dịch bit tính toán hệ số lượng tử: $\text{shift} = 2^{-i}$ (thực hiện dịch phải i bit trong phần cứng).
- **Bước 3 (Cập nhật ma trận xoay Hyperbolic song song):**
 - Tính toán giá trị mới: $x_n = x + d \cdot y \cdot 2^{-i}$ và $y_n = y + d \cdot x \cdot 2^{-i}$.
 - Chốt đồng bộ dữ liệu vào thanh ghi: $x = x_n$ và $y = y_n$.
- **Bước 4:** Thực hiện tăng chỉ số bộ đếm $i++$ và quay ngược lên khối kiểm tra điều kiện dừng $i = 32$?

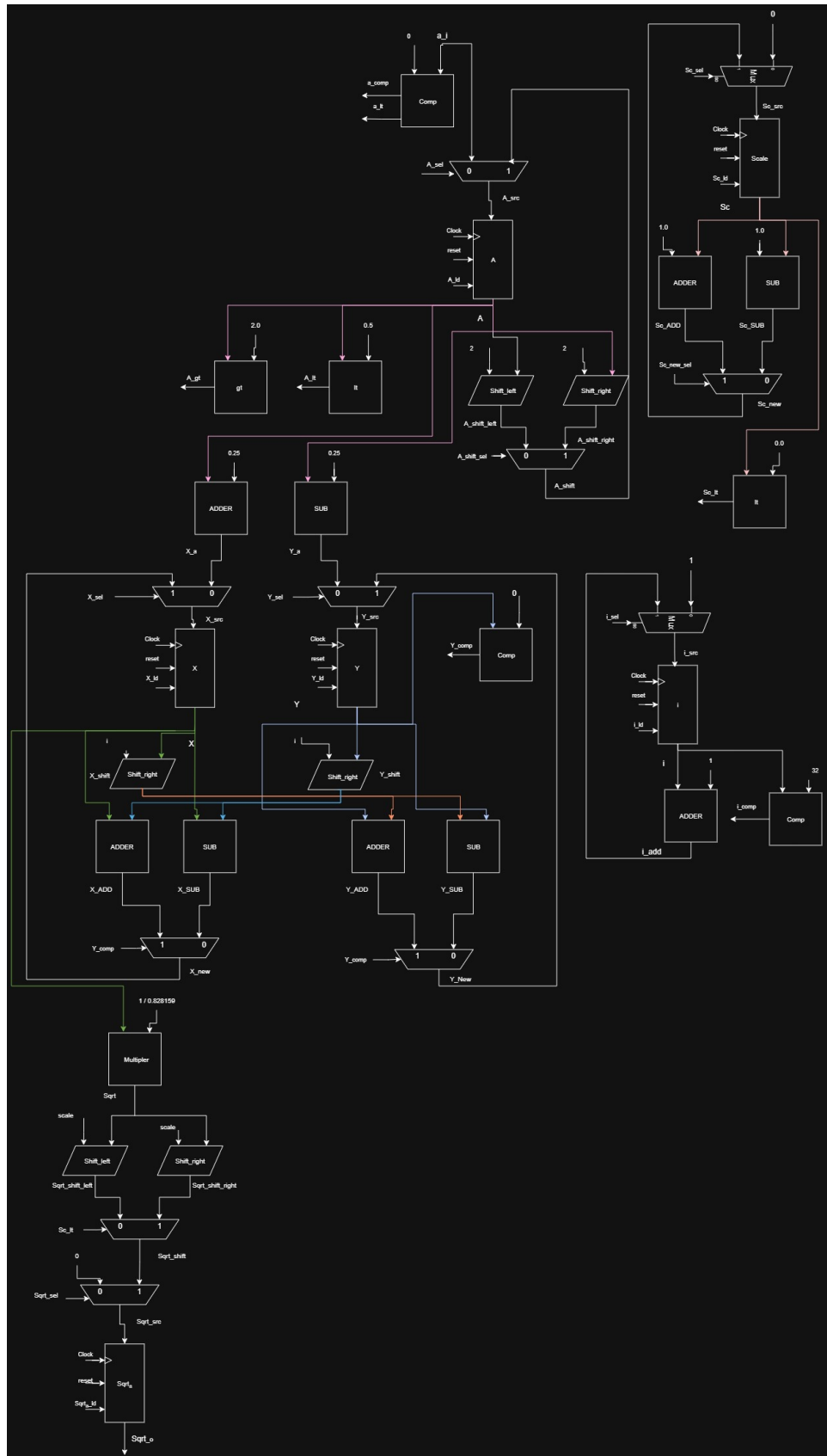
Giai đoạn 5 Khử hệ số co giãn và Tái chuẩn hóa đầu ra Khi vòng lặp CORDIC kết thúc thành công, biến y được triệt tiêu về xấp xỉ bằng 0. Thanh ghi x lúc này chứa giá trị căn bậc hai nhân với hệ số co giãn cấu trúc K .

- **Bước 1:** Hệ thống tiến hành chia giá trị trong thanh ghi x cho hằng số co giãn hội tụ để thu được giá trị căn cơ sở: $\text{sqrt_a} = x/0.828159$ (Trong mạch thực tế thường thực hiện bằng phép nhân với số nghịch đảo $\frac{1}{K} \approx 1.2075$).
- **Bước 2 (Tái chuẩn hóa theo Scale gốc):** Để bù lại việc ta đã dịch chuyển dữ liệu gốc a theo cơ số 4^{Scale} ở giai đoạn 2, kết quả căn bậc hai sẽ được nhân trả lại với cơ số 2^{Scale} .

$$\text{sqrt_a} = \text{sqrt_a} \cdot 2^{\text{Scale}}$$

- **Bước 3:** Hệ thống kích hoạt chân cờ báo hiệu đầu ra $\text{Done} = '1'$ nhằm xác nhận dữ liệu đã tính toán xong và hoàn toàn ổn định trên Bus hệ thống. Ngay sau đó, FSM điều khiển luồng quay trở lại trạng thái **Idle** ban đầu để chuẩn bị tiếp nhận một chu kỳ tính toán mới từ chân Start .

4.2. Đơn vị xử lý dữ liệu (Datapath)



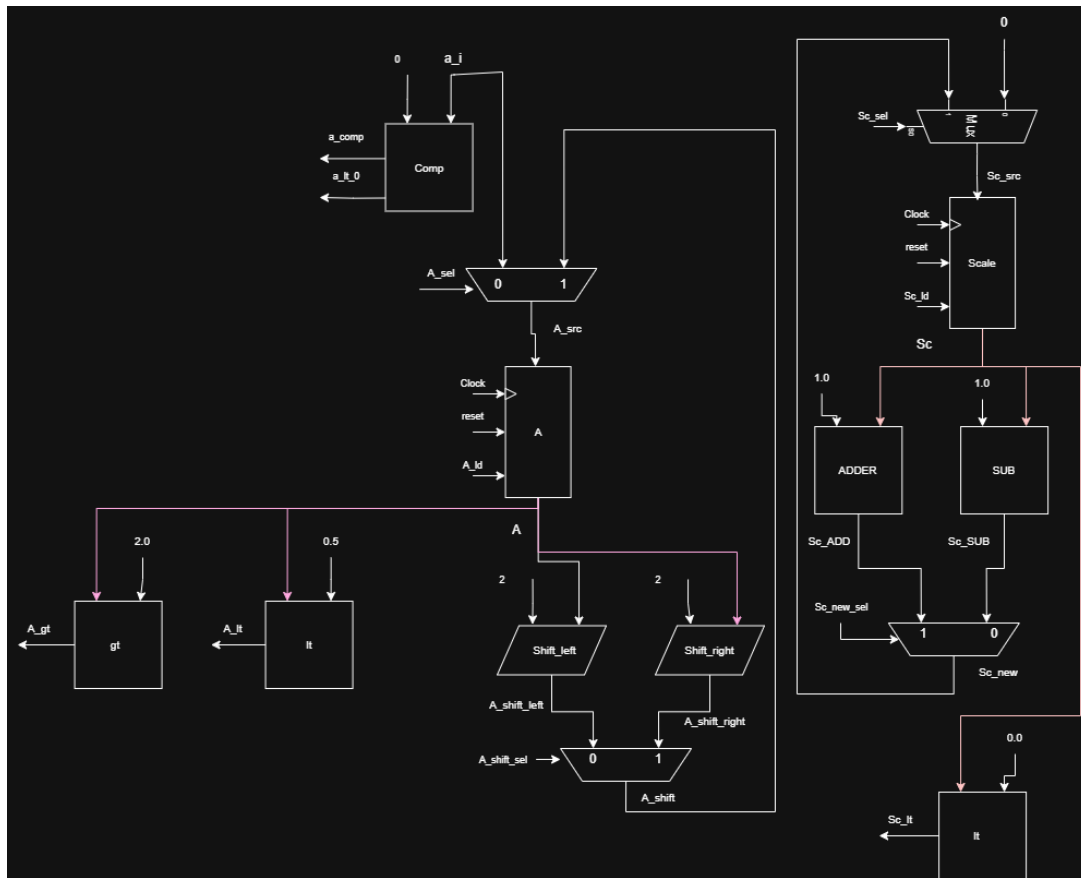
Hình 3: Cấu trúc của đơn vị xử lý dữ liệu Datapath

Bảng 1: Mô tả các tín hiệu vào/ra của khối datapath

TT	Port	Direction	Width	Meaning
1	CLK	IN	1	Cấp xung clock hoạt động cho mạch
2	RST	IN	1	Reset mạch về trạng thái ban đầu
3	a_i	IN	16	Tín hiệu đầu vào (số cần tính căn bậc hai)
4	a_lt_0	OUT	1	Tín hiệu đầu ra của bộ so sánh $a < 0$
5	a_comp	OUT	1	Tín hiệu đầu ra của bộ so sánh $a = 0$
6	A_ld	IN	1	Tín hiệu cho phép load vào thanh ghi A
7	A_shift_sel	IN	1	Tín hiệu chọn cho tín hiệu A_shift
8	A_sel	IN	1	Tín hiệu chọn lối vào cho thanh ghi A
9	A_lt	OUT	1	Tín hiệu lối ra tại bộ so sánh A với 0.5
10	A_gt	OUT	1	Tín hiệu lối ra tại bộ so sánh A với 2.0
11	Sc_sel	IN	1	Tín hiệu chọn lối vào cho thanh ghi Sc
12	Sc_ld	IN	1	Tín hiệu cho phép load vào thanh ghi Sc
13	Sc_new_sel	IN	1	Tín hiệu chọn cho tín hiệu Sc_new
14	Sc_lt	OUT	1	Tín hiệu lối ra của bộ so sánh Sc với 1
15	i_ld	IN	1	Tín hiệu cho phép load vào thanh ghi i
16	i_sel	IN	1	Tín hiệu chọn cho tín hiệu i_src
17	i_comp	OUT	1	Tín hiệu lối ra của bộ so sánh i với 32
18	Y_comp	OUT	1	Tín hiệu lối ra của bộ so sánh Y với 0
19	Sqrt_sel	IN	1	Tín hiệu chọn lối vào cho thanh ghi Sqrt
20	Sqrt_ld	IN	1	Tín hiệu cho phép load vào thanh ghi Sqrt
21	Sqrt_o	OUT	16	Tín hiệu lối ra thanh ghi Sqrt
22	X_ld	IN	1	Tín hiệu cho phép load vào thanh ghi X
23	X_sel	IN	1	Tín hiệu chọn lối vào cho thanh ghi X
24	Y_ld	IN	1	Tín hiệu cho phép load vào thanh ghi Y
25	Y_sel	IN	1	Tín hiệu chọn lối vào cho thanh ghi Y

Hoạt động của datapath: Cấu trúc của datapath gồm 3 khối: **Khối chuẩn hóa đầu vào a**, **Khối lập và tính toán giá trị X, Y** và **Khối tính toán kết quả**.

Khối chuẩn hóa lỗi vào a về $[0.5, 2]$

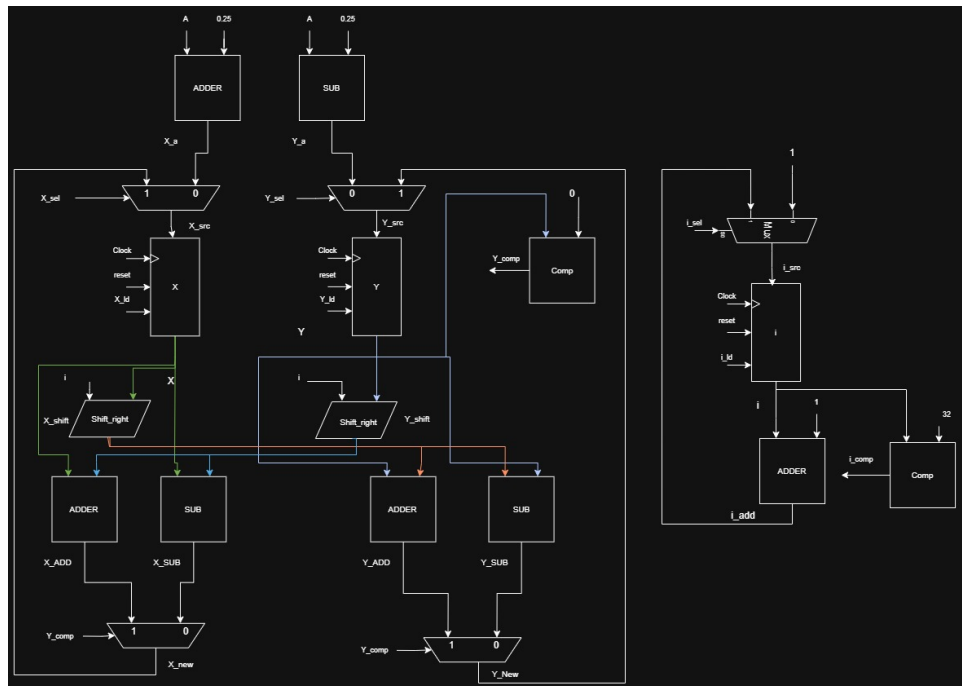


Hình 4: Khối chuẩn hóa lỗi vào a_i

1. **a_i** đầu vào sẽ được so sánh với 0.
 - Nếu **a_i** < 0 thì tín hiệu **a_lt_0** = '1', báo cho FSM biết đầu vào a < 0, FSM báo có err vì số âm không thể tính căn bậc hai.
 - Nếu **a_i** = 0 thì tín hiệu **a_comp** = '1'
2. Tại bộ mux **A**, tín hiệu lỗi vào **A_sel** sẽ quyết định tín hiệu nào sẽ được nạp vào thanh ghi **A**
 - Nếu **A_sel** = '1', tín hiệu **A_shift** sẽ đi vào lỗi vào thanh ghi **A**.
 - Nếu **A_sel** = '0', tín hiệu **a_i** sẽ đi vào thanh ghi **A**.
 - **A_src** nối với thanh ghi **A**
3. Tại lỗi ra của thanh ghi **A**, tín hiệu **A** đi qua:
 - Một bộ so sánh < 0.5: Tín hiệu **A_lt** = '1' khi $A < 0.5$ và ngược lại '0'.
 - Một bộ so sánh ≥ 2.0 : Tín hiệu **A_gt** = '1' khi $A \geq 2.0$ và ngược lại '0'.
 - Một bộ dịch trái 2 bit **Shift_left**: Tín hiệu **A_shift_left** sẽ được gán bằng tín hiệu **A** sau khi dịch trái 2 bit.

- Một bộ dịch phải 2 bit **Shift_left**: Tín hiệu **A_shift_right** sẽ được gán bằng tín hiệu **A** sau khi dịch phải 2 bit.
 - FSM nhận 2 tín hiệu **A_gt** và **A_lt** để lặp tại cho a cho đến khi a trong khoảng $\{0.5; 2\}$
4. Tại bộ mux **A_shift**, tín hiệu lỗi vào **A_shift_sel** sẽ quyết định tín hiệu **A_shift** sẽ được gán bằng tín hiệu nào.
 - Nếu **A_shift_sel** = '1', tín hiệu **A_shift** sẽ được gán bằng tín hiệu **A_shift_right**.
 - Nếu **A_shift_sel** = '0', tín hiệu **A_shift** sẽ được gán bằng tín hiệu **A_shift_left**.
 5. Trong khi đó, tại khối tính toán **Scale**, tín hiệu lỗi vào **Sc_sel** sẽ quyết định tín hiệu tín hiệu nào sẽ được nạp vào thanh ghi **Scale**
 - Nếu **Sc_sel** = '1', tín hiệu giá trị mới **Sc_new** sẽ được nạp vào thanh ghi **Scale**.
 - Nếu **Sc_sel** = '0', tín hiệu '0' sẽ được nạp vào thanh ghi **Scale**.
 6. Tại lối ra của thanh ghi **Scale**, tín hiệu **Sc** đi qua:
 - Một bộ **ADDER**: tín hiệu sẽ được cộng thêm 1 và đi vào bộ mux.
 - Một bộ **SUB**: tín hiệu sẽ được trừ đi 1 và đi vào bộ mux.
 - Một bộ so sánh < 0.0 : Tín hiệu **Sc_lt** = '1' khi $Sc < 0$ và ngược lại '0'.
 7. Tại bộ mux **Sc_new**, tín hiệu lỗi vào **Sc_new_sel** sẽ quyết định sẽ quyết định tín hiệu **Sc_new** sẽ được gán bằng tín hiệu nào.
 - Nếu **Sc_new_sel** = '1', tín hiệu **Sc_new** sẽ được gán bằng tín hiệu **Sc_ADD**.
 - Nếu **Sc_new_sel** = '0', tín hiệu **Sc_new** sẽ được gán bằng tín hiệu **Sc_SUB**.
 - Kết quả của **Sc_lt** là dương hay âm sẽ quyết định dịch trái hay dịch phải

Khối lặp và tính toán X, Y



Hình 5: Khối tính toán X, Y và khối lặp

1. Khởi tạo các giá trị X, Y

- Tín hiệu A sau khi được chuẩn hóa về khoảng $[0.5, 2]$:
 - (a) Tín hiệu **X_a** sẽ được gán bằng giá trị $A + 0.25$.
 - (b) Tín hiệu **Y_a** sẽ được gán bằng giá trị $A - 0.25$.

2. Tại bộ mux **X**, tín hiệu lỗi vào **X_sel** sẽ quyết định tín hiệu nào sẽ được load vào thanh ghi **X**.

- **X_sel**: Nếu bằng 1, chọn ngõ vào từ bên ngoài; nếu bằng 0, chọn **X_new** (kết quả của vòng lặp trước). Đầu ra là **X_src**.
- **Y_sel**: Nếu bằng 1, chọn ngõ vào từ bên ngoài; nếu bằng 0, chọn **Y_new**. Đầu ra là **Y_src**.

3. Tại lối ra thanh ghi **X**, tín hiệu **X** sẽ đi qua:

- Một bộ dịch **SHIFT**: Tín hiệu **X** sẽ được dịch 1 khoảng i bit (i là giá trị trong bộ lặp) và đi vào 2 bộ **ADD** và **SUB** để tạo tín hiệu **Y_new**.
- Hai bộ **ADD**, **SUB**: Tín hiệu **X** sẽ cùng tín hiệu **Y_shift** đi vào hai bộ cộng trừ để tạo ra hai tín hiệu **X_ADD** và **X_SUB**.

4. Tương tự với giá trị **X**, tín hiệu **Y** được đi qua các đường dẫn có cấu trúc tương tự.

5. Tại bộ mux **Y**, tín hiệu lỗi vào **Y_sel** sẽ quyết định tín hiệu nào sẽ được load vào thanh ghi **Y**.

- Nếu $Y_sel = '0'$ tín hiệu Y_a sẽ đi vào lối vào thanh ghi Y .
- Nếu $Y_sel = '1'$ tín hiệu Y_new sẽ đi vào lối vào thanh ghi Y .

6. Tại lối ra thanh ghi Y , tín hiệu Y sẽ đi qua:

- Một bộ so sánh < 0 : Tín hiệu $Y_comp = '1'$ khi $Y < 0$ và ngược lại $'0'$.
- Một bộ dịch **SHIFT**: Tín hiệu Y sẽ được dịch 1 khoảng i bit (i là giá trị trong bộ lặp) và đi vào 2 bộ **ADD** và **SUB** để tạo tín hiệu X_new .
- Hai bộ **ADD**, **SUB**: Tín hiệu Y sẽ cùng tín hiệu X_shift đi vào hai bộ cộng trừ để tạo ra hai tín hiệu Y_ADD và Y_SUB .

7. Tại bộ mux X_new , tín hiệu Y_comp sẽ quyết định tín hiệu X_new sẽ được gán bằng tín hiệu nào.

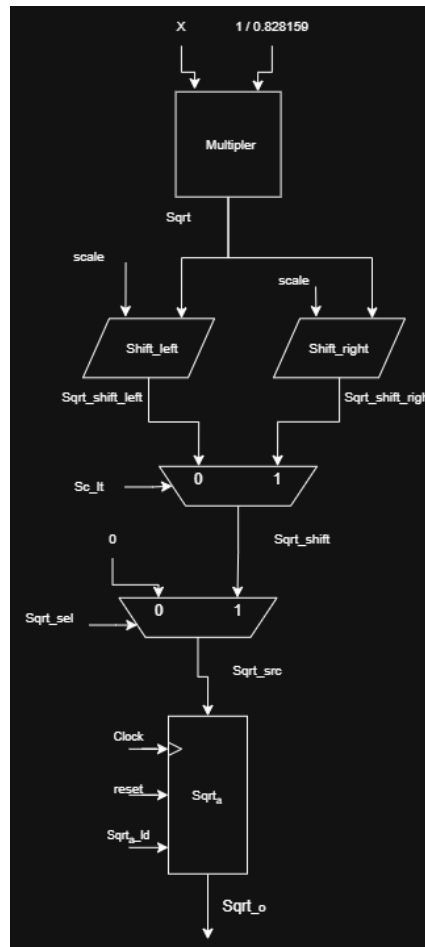
- Nếu $Y_comp = '1'$ tín hiệu X_new sẽ được gán X_ADD .
- Nếu $Y_comp = '0'$ tín hiệu X_new sẽ được gán X_SUB .

8. Tại bộ mux Y_new , tín hiệu Y_comp sẽ quyết định tín hiệu Y_new sẽ được gán bằng tín hiệu nào.

- Nếu $Y_comp = '1'$ tín hiệu Y_new sẽ được gán Y_ADD .
- Nếu $Y_comp = '0'$ tín hiệu Y_new sẽ được gán Y_SUB .

9. Tại khối lặp, tín hiệu i liên tục được cộng 1 và so sánh với 32 (số vòng lặp) nếu giá trị của tín hiệu i bằng 32 thì tín hiệu $i_comp = '1'$.

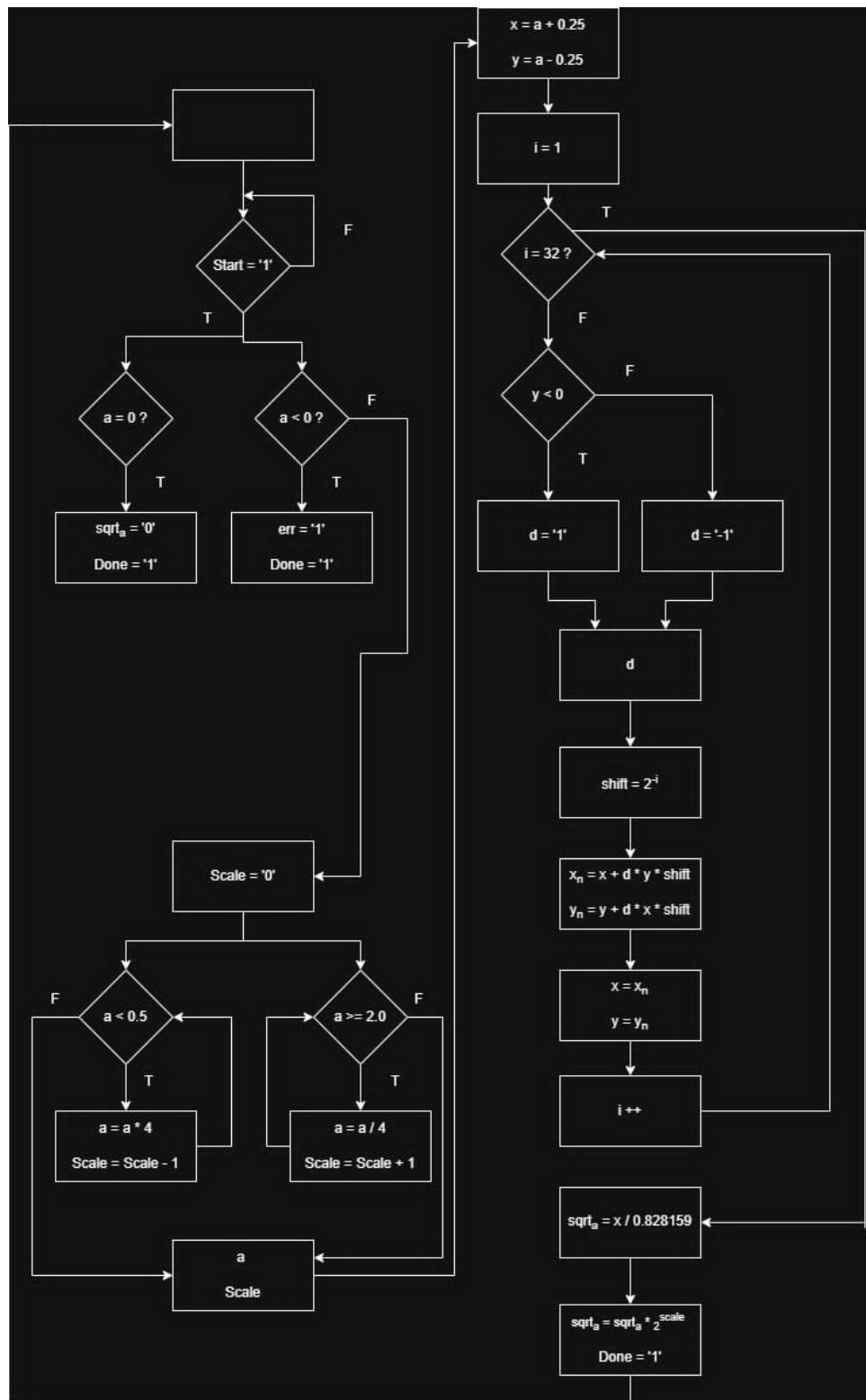
Khối tính toán kết quả



Hình 6: Khối tính toán kết quả

1. Tín hiệu **X** sau khi lặp lại 32 lần tính toán sẽ được đưa vào 1 bộ nhân cùng với hệ số $\frac{1}{K}$ (trong đó, **K** là hệ số khuếch đại, hội tụ về 0.828159). Sau đó, kết quả của phép nhân chính là tín hiệu **Sqrt**.
2. Tín hiệu **Sqrt** sau đó được đưa qua hai bộ dịch trái phải để dịch với số bit dịch bằng với tỉ lệ **Scale** đã tính toán ở Khối chuẩn hóa đầu vào a.
3. Tại bộ mux **Sqrt_shift**, tín hiệu **Sc_lt** sẽ quyết định tín hiệu **Sqrt_shift** sẽ được gán bằng tín hiệu nào.
 - Nếu **Sc_lt** = '1' tín hiệu **Sqrt_shift** sẽ được gán **Sqrt_shift_right**.
 - Nếu **Sc_lt** = '0' tín hiệu **Sqrt_shift** sẽ được gán **Sqrt_shift_left**.
4. Tại bộ mux **Sqrt**, tín hiệu lỗi vào **Sqrt_sel** sẽ quyết định tín hiệu đầu vào của thanh ghi **Sqrt** sẽ là tín hiệu nào.
 - Nếu **Sqrt_sel** = '1' tín hiệu **Sqrt_shift** sẽ đi vào lỗi vào của thanh ghi **Sqrt**.
 - Nếu **Sc_lt** = '0' tín hiệu '0' sẽ đi vào lỗi vào của thanh ghi **Sqrt** chờ được nạp.

4.3.Đơn vị điều khiển (Control Unit)



Hình 7: Máy FSM của đơn vị điều khiển.

Control Unit bao gồm 16 trạng thái như sau:

Bảng 2: Bảng mô tả trạng thái của đơn vị điều khiển

Tên trạng thái	Chức năng	Tín hiệu lỗi ra
S0	Trạng thái khởi tạo	
S1	Trạng thái kiểm tra tín hiệu <i>Start_i</i>	
S2	Trạng thái khi lỗi vào bằng 0	<i>Sqrt_sel</i> = '0' <i>Sqrt_ld</i> = '1'
S3	Trạng thái khi lỗi vào là số nhỏ hơn 0	<i>err</i> = '1'
S4	Trạng thái khởi tạo <i>Scale</i>	<i>Sc_sel</i> = '0' <i>Sc_ld</i> = '1'
S5	Trạng thái nạp thanh ghi A	<i>A_sel</i> = '0' <i>A_ld</i> = '1'
S6	Trạng thái kiểm tra A	
S7	Trạng thái khi $A < 0.5$ ($A = A \ll 2$) (<i>scale--</i>)	<i>A_shift_sel</i> = '0' <i>Sc_sel</i> = '1' <i>Sc_ld</i> = '1' <i>Sc_new_sel</i> = '0' <i>A_sel</i> = '1' <i>A_ld</i> = '1'
S8	Trạng thái khi $A \geq 2.0$ ($A = A \gg 2$) (<i>scale++</i>)	<i>A_shift_sel</i> = '1' <i>Sc_sel</i> = '1' <i>Sc_ld</i> = '1' <i>Sc_new_sel</i> = '1' <i>A_sel</i> = '1' <i>A_ld</i> = '1'
S9	Trạng thái khởi tạo X, Y	<i>X_sel</i> = '0' <i>X_ld</i> = '1' <i>Y_sel</i> = '0' <i>Y_ld</i> = '1'

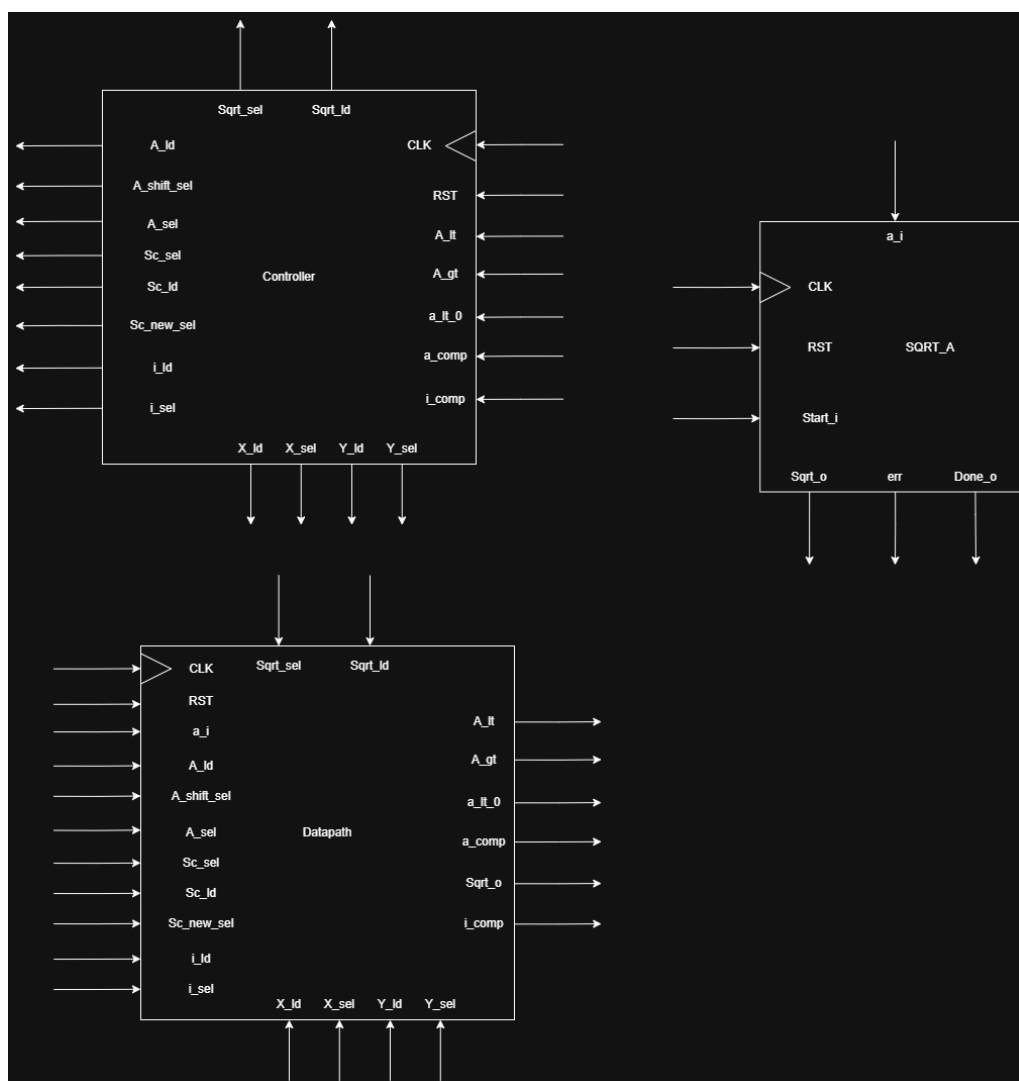
Tên trạng thái	Chức năng	Tín hiệu lỗi ra
S10	Trạng thái khởi tạo i	$i_sel = '0'$ $i_ld = '1'$
S11	Trạng thái kiểm tra $i < 32$?	
S12	Trạng thái kiểm tra $Y < 0$?	
S13	Trạng thái cập nhật giá trị mới cho X và Y	$X_sel = '1'$ $X_ld = '1'$ $Y_sel = '1'$ $Y_ld = '1'$
S14	Trạng thái tăng i ($i++$)	$i_sel = '1'$ $i_ld = '1'$
S15	Trạng thái cập nhật $Sqrt$	$Sqrt_sel = '1'$ $Sqrt_ld = '1'$
S16	Trạng thái hoàn thành	$Done_o = '1'$

Hoạt động chuyển trạng thái của **Control Unit**:

1. Tại thời điểm khởi tạo (S0), nếu tín hiệu **Reset** = '1', tiếp tục giữ nguyên trạng thái.
2. Khi tín hiệu **Reset** = '0' thì từ trạng thái S0 sẽ chuyển sang trạng thái S1.
3. Tại trạng thái S1:
 - (a) Nếu tín hiệu **Start_i** = '0', trạng thái vẫn giữ nguyên tại S1.
 - (b) Nếu tín hiệu **Start_i** = '1':
 - i. Kiểm tra tín hiệu **a_comp** và **a_lt_0**.
 - A. Nếu tín hiệu **a_comp** = '1', trạng thái sẽ chuyển sang S2.
 - B. Nếu tín hiệu **a_lt_0** = '1', trạng thái sẽ chuyển sang S3.
 - ii. Nếu cả hai tín hiệu **a_comp** và **a_lt_0** đều bằng '0' và các trường hợp còn lại, trạng thái sẽ chuyển sang S4.
4. Tại trạng thái S2 hay S3, trạng thái đều sẽ chuyển sang trạng thái S16.
5. Tại trạng thái S4, trạng thái sẽ chuyển sang S5.
6. Tại trạng thái S5, trạng thái sẽ chuyển sang S6.
7. Tại trạng thái S6:

- (a) Nếu tín hiệu $A_{lt} = '1'$, trạng thái sẽ chuyển sang S7.
 - (b) Nếu tín hiệu $A_{gt} = '1'$, trạng thái sẽ chuyển sang S8.
 - (c) Nếu tín hiệu A_{lt} và A_{gt} đều bằng '0' và các trường hợp còn lại, trạng thái sẽ chuyển sang S9.
8. Tại trạng thái S7 hay S8 trạng thái cũng sẽ đều chuyển về trạng thái S6.
 9. Tại trạng thái S9, trạng thái sẽ chuyển sang S10.
 10. Tại trạng thái S10, trạng thái sẽ chuyển sang S11.
 11. Tại trạng thái S11:
 11. Tại trạng thái SI1:
 - [a.]
 - (a) Nếu $i_{comp} = '1'$, tức là giá trị i đã đạt tới 32, thì trạng thái sẽ chuyển tới trạng thái SI5.
 - (b) Ngược lại thì trạng thái sẽ chuyển tiếp tới SI2.
 12. Tại trạng thái SI2, trạng thái sẽ chuyển sang SI3.
 13. Tại trạng thái SI3, trạng thái sẽ chuyển sang SI4.
 14. Tại trạng thái SI4, trạng thái sẽ chuyển sang SI1.
 15. Tại trạng thái SI5, trạng thái sẽ chuyển sang SI6.
 16. Tại trạng thái SI6, trạng thái sẽ chuyển sang S0.
 17. Các trạng thái còn lại cũng sẽ chuyển về S0.

4.4. Sơ đồ khối tổng thể



Hình 8: Sơ đồ khối tổng thể của bộ tính hàm căn bậc hai.

Mô hình hóa bằng VHDL

Tổng quan cấu trúc dự án

Dự án được phân chia theo mô hình **FSMD (Finite State Machine with Datapath)**, một phương pháp thiết kế phần cứng chuẩn mực cho các hệ thống số xử lý thuật toán phức tạp. Cấu trúc hệ thống bao gồm 11 module được tổ chức phân cấp rõ ràng:

- **Module Top-level (SQRT.vhd):** Đóng vai trò kết nối mạch lạc giữa Khối Điều khiển (Control Unit) và Khối Dữ liệu (Datapath)[cite: 81].
- **Khối Điều khiển (controller.vhd):** Là một máy trạng thái hữu hạn (FSM) quản lý tiến trình và các bước thực thi luân phiên của thuật toán tính căn bậc hai[cite: 1].

- **Khối Dữ liệu (datapath.vhd):** Thực hiện toàn bộ các phép toán số học như cộng, trừ, nhân, dịch bit và lưu trữ dữ liệu tạm thời thông qua hệ thống các thanh ghi[cite: 50].
- **Các module hỗ trợ (Sub-modules):** Gồm bộ cộng (ADDER), bộ trừ (SUBT), bộ nhân (Multiplier), mạch dịch trái/phải (Shift_l, Shift_r), và thanh ghi n-bit tổng quát (regn). Các thành phần này được khai báo và quản lý tập trung trong package Mylib.

4.2 Phân tích chi tiết các Module chính và Giao tiếp (Interface)

4.3 Module top: SQRT

- **Chức năng:** Là thực thể ở cấp cao nhất (Top-level Entity), bao bọc toàn bộ hệ thống tính toán căn bậc hai thành một khối chức năng duy nhất, giúp dễ dàng tích hợp vào các hệ thống lớn hơn[cite: 81].
- **Giao tiếp (Interface):**
 - Hỗ trợ tham số hóa độ rộng dữ liệu thông qua hằng số Generic DATA_WIDTH (mặc định là 16-bit)[cite: 82].
 - **Tín hiệu đầu vào:** CLK (Xung nhịp hệ thống), RST (Tín hiệu khởi động lại hệ thống - hoạt động ở mức cao), Start_i (Tín hiệu kích hoạt phép tính), và a_i (Dữ liệu đầu vào cần tính căn)[cite: 82].
 - **Tín hiệu đầu ra:** Sqrt_o (Dữ liệu kết quả căn bậc hai), Done_o (Tín hiệu cờ báo hiệu đã hoàn thành tính toán), và err (Cờ báo lỗi khi dữ liệu đầu vào không hợp lệ như số âm)[cite: 83].
- **Kiến trúc (Architecture):** Sử dụng kỹ thuật cấu trúc (structural) để liên kết thực thể điều khiển controller (CU) và thực thể dữ liệu datapath (DA) thông qua các đường tín hiệu nội bộ.

4.4 Khối Điều khiển: Controller

- **Chức năng:** Quản lý thứ tự thực hiện thuật toán bằng một Máy trạng thái hữu hạn (FSM) gồm 17 trạng thái (từ S0 đến S16)[cite: 1].
- **Giao tiếp (Interface):**
 - **Đầu vào:** Các cờ trạng thái logic phản hồi từ Datapath (a_comp, a_lt_0, A_gt, A_lt, i_comp)[cite: 2].
 - **Đầu ra:** Các tín hiệu điều khiển chọn kênh (_sel) cho các bộ đa kênh (Mux) và tín hiệu cho phép nạp dữ liệu (_ld) tới các thanh ghi lưu trữ trong Datapath[cite: 3, 4].

4.5 Khối Dữ liệu: Datapath

- **Chức năng:** Thực hiện xử lý toán học trực tiếp trên các luồng dữ liệu dưới sự điều khiển của FSM[cite: 50].

- Quy trình và các khối chính bên trong:

1. **Kiểm tra điều kiện:** So sánh trực tiếp giá trị ngõ vào a_i với số 0 để xuất các cờ báo hiệu ban đầu.
2. **Chuẩn hóa dữ liệu (Normalization):** Sử dụng bộ dịch $Shift_l$ và $Shift_r$ dịch chuyển bit của số A để đưa giá trị về vùng hội tụ chuẩn ($0.5 \leq A < 2.0$), đồng thời bộ đếm Sc (Scale) sẽ lưu lại độ lệch dịch chuyển này[cite: 56, 58].
3. **Vòng lặp toán học:** Chạy vòng lặp tối đa 32 chu kỳ. Tại mỗi chu kỳ, tùy thuộc vào dấu của biến Y (Y_comp), hệ thống quyết định cộng hoặc trừ chéo giá trị dịch bit của X và Y thông qua $ADDER$ và $SUBT$.
4. **Hiệu chỉnh và xuất kết quả:** Kết quả sau vòng lặp được nhân với hằng số tỉ lệ $1/K$ bằng bộ nhân $Multiplier$, sau đó dịch ngược lại dựa trên giá trị lưu trong thanh ghi Sc để tái thiết lập đúng dấu phẩy động cố định trước khi xuất ra $Sqrt_o$.

4.6 Mã nguồn VHDL Khối SQRT (Top-level)

```

1  LIBRARY ieee;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use work.Mylib.all;
5
6  ENTITY SQRT IS
7      GENERIC(DATA_WIDTH: INTEGER := 16);
8      PORT(
9          CLK, RST, Start_i : IN STD_LOGIC;
10         a_i                : IN STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
11         Done_o, err        : OUT STD_LOGIC;
12         Sqrt_o             : OUT STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0)
13     );
14 END SQRT;
15
16 ARCHITECTURE arc of SQRT IS
17     SIGNAL A_ld: STD_LOGIC;
18     SIGNAL Sc_sel, Sc_ld, Sc_new_sel, i_sel, i_ld: STD_LOGIC;
19     SIGNAL A_sel, X_sel, Y_sel, X_ld, Y_ld: STD_LOGIC;
20     SIGNAL X_shift_sel, Y_shift_sel, X_new_sel, Y_new_sel, A_shift_sel:
STD_LOGIC;
21     SIGNAL Sqrt_shift_sel, Sqrt_ld, Sqrt_sel : STD_LOGIC;
22
23     SIGNAL a_comp : STD_LOGIC;
24     SIGNAL a_lt_0 : STD_LOGIC;
25     SIGNAL A_gt, A_lt, Sc_lt: STD_LOGIC;
26     SIGNAL Y_comp : STD_LOGIC;
27     SIGNAL i_comp : STD_LOGIC;
28 Begin
29
30     -- (Controller Unit)
31     CU: controller

```



```

32  PORT MAP(
33      RST          => RST,
34      CLK          => CLK,
35      Start_i      => Start_i,
36      a_comp       => a_comp,
37      a_lt_0       => a_lt_0,
38      A_gt         => A_gt,
39      A_lt         => A_lt,
40      i_comp       => i_comp,
41      A_ld         => A_ld,
42      Sc_sel       => Sc_sel,
43      Sc_ld        => Sc_ld,
44      Sc_new_sel   => Sc_new_sel,
45      i_sel        => i_sel,
46      i_ld         => i_ld,
47      A_sel        => A_sel,
48      X_sel        => X_sel,
49      Y_sel        => Y_sel,
50      X_ld         => X_ld,
51      Y_ld         => Y_ld,
52      A_shift_sel  => A_shift_sel,
53      Sqrt_ld      => Sqrt_ld,
54      Sqrt_sel     => Sqrt_sel,
55      Done_o       => Done_o,
56      err          => err
57  );
58
59  -- (Datapath)
60  DA: datapath
61  GENERIC MAP(DATA_WIDTH)
62  PORT MAP(
63      CLK          => CLK,
64      RST          => RST,
65      a_i          => a_i,
66      A_ld         => A_ld,
67      Sc_sel       => Sc_sel,
68      Sc_ld        => Sc_ld,
69      i_sel        => i_sel,
70      i_ld         => i_ld,
71      A_shift_sel  => A_shift_sel,
72      Sc_new_sel   => Sc_new_sel,
73      A_sel        => A_sel,
74      X_sel        => X_sel,
75      Y_sel        => Y_sel,
76      X_ld         => X_ld,
77      Y_ld         => Y_ld,
78      Sqrt_ld      => Sqrt_ld,
79      Sqrt_sel     => Sqrt_sel,
80      Sqrt_o       => Sqrt_o,
81      a_comp       => a_comp,
82      a_lt_0       => a_lt_0,

```

```

83         A_gt          => A_gt,
84         A_lt          => A_lt,
85         i_comp        => i_comp
86     );
87
88 End arc;

```

Listing 1: Nội dung tệp cấu trúc hệ thống SQRT.vhd

4.7 Mã nguồn VHDL Khối Datapath

Dưới đây là nội dung toàn vẹn của tệp datapath.vhd, mô tả chi tiết đường dữ liệu phần cứng để thực hiện thuật toán tính căn bậc hai.

```

1  Library ieee;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use work.Mylib.all;
5
6  Entity datapath IS
7      GENERIC(DATA_WIDTH : INTEGER := 16);
8      PORT(
9          CLK, RST          : IN STD_LOGIC;
10         a_i               : IN STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
11         A_ld              : IN STD_LOGIC;
12         Sc_sel, Sc_ld, i_sel, i_ld : IN STD_LOGIC;
13         A_shift_sel, Sc_new_sel, A_sel, X_sel, Y_sel, X_ld, Y_ld : IN
STD_LOGIC;
14         Sqrt_ld, Sqrt_sel : IN STD_LOGIC;
15
16         Sqrt_o            : OUT STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
17         a_comp            : OUT STD_LOGIC;
18         a_lt_0            : OUT STD_LOGIC;
19         A_gt, A_lt        : OUT STD_LOGIC;
20         i_comp            : OUT STD_LOGIC
21     );
22 end datapath;
23
24 ARCHITECTURE arc of datapath IS
25     SIGNAL A_shift, A_shift_right, X_ADD, X_SUB, Y_ADD, Y_SUB, A_shift_left :
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
26     SIGNAL X_shift_right, Y_shift_right, A_src, X_src, Y_src, X_a, Y_a :
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
27     SIGNAL A, X, Y, X_new, Y_new, X_shift, Y_shift, Sqrt, Sqrt_src :
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
28     SIGNAL Sqrt_shift, Sqrt_shift_right, Sqrt_shift_left, K :
STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
29
30     SIGNAL i_src, i, i_add, Sc_src, Sc, Sc_new, Sc_ADD, Sc_SUB :
STD_LOGIC_VECTOR(5 downto 0);
31     SIGNAL shift_val_1, shift_val_2 : integer range 0 to 32;

```

```

32     SIGNAL Sc_lt, Y_comp : STD_LOGIC;
33 Begin
34     -- comp a_i vs 0
35     a_comp <= '1' when (signed(a_i) = to_signed(0,DATA_WIDTH)) else '0';
36     a_lt_0 <= '1' when (signed(a_i) < to_signed(0,DATA_WIDTH)) else '0';
37
38     -- mux A
39     A_src <= a_i when A_sel = '0' else A_shift;
40
41     -- shift A
42     Shift_right_A: Shift_r
43     GENERIC MAP(DATA_WIDTH)
44     port map(
45         DIN    => A,
46         SHIFT  => 2,
47         DOUT   => A_shift_right
48     );
49
50     Shift_left_A: Shift_l
51     GENERIC MAP(DATA_WIDTH)
52     port map(
53         DIN    => A,
54         SHIFT  => 2,
55         DOUT   => A_shift_left
56     );
57
58     A_shift <= A_shift_left when A_shift_sel = '0' else A_shift_right;
59
60     -- regA
61     RegA : regn
62     Generic MAP (DATA_WIDTH)
63     PORT map (RST, CLK, A_ld, A_src, A);
64
65     -- so snh A vi 0.5 v 2.0
66     A_lt <= '1' when (signed(A) < to_signed(8192, 16)) else '0';
67
68     -- A >= 2.0
69     A_gt <= '1' when (signed(A) >= to_signed(32767, 16)) else '0';
70
71     -- scale
72     Sc_src <= Sc_new when (Sc_sel = '1') else (others => '0');
73
74     -- reg Scale
75     RegSc : regn
76     Generic MAP (6)
77     PORT map (RST, CLK, Sc_ld, Sc_src, Sc);
78
79     -- add/sub scale
80     Sc_ADD <= std_logic_vector(signed(Sc) + 1);
81     Sc_SUB <= std_logic_vector(signed(Sc) - 1);
82     Sc_new <= Sc_SUB when Sc_new_sel = '0' else Sc_ADD;

```

```

83
84 Sc_lt <= '1' when signed(Sc) < to_signed(0,6) else '0';
85
86 -- x = a + 0.25, y = a - 0.25
87 X_a <= std_logic_vector(signed(A) + to_signed(4096, 16));
88 Y_a <= std_logic_vector(signed(A) - to_signed(4096, 16));
89 X_src <= X_a when X_sel = '0' else X_new;
90 Y_src <= Y_a when Y_sel = '0' else Y_new;
91
92 -- regX
93 RegX : regn
94   Generic MAP (DATA_WIDTH)
95   PORT map (RST, CLK, X_ld, X_src, X);
96
97 -- regY
98 RegY : regn
99   Generic MAP (DATA_WIDTH)
100  PORT map (RST, CLK, Y_ld, Y_src, Y);
101
102 -- i
103 i_src <= "000001" when (i_sel = '0') else i_add;
104
105 -- reg i
106 Regi : regn
107   Generic MAP (6)
108   PORT map (RST, CLK, i_ld, i_src, i);
109
110 -- add i
111 i_add <= std_logic_vector(unsigned(i) + 1);
112
113 -- comp i_add vs 32
114 i_comp <= '1' when (unsigned(i) = 32) else '0';
115
116 -- comp Y vs 0 <0 +, >0 -
117 Y_comp <= '1' when (signed(Y) < 0 ) else '0';
118 shift_val_1 <= to_integer(unsigned(i));
119 shift_val_2 <= abs(to_integer(signed(Sc)));
120
121 -- shift X
122 Shift_right_X: Shift_r
123   GENERIC MAP(DATA_WIDTH)
124   port map(
125     DIN    => X,
126     SHIFT => shift_val_1,
127     DOUT   => X_shift
128   );
129
130 -- shift Y
131 Shift_right_Y: Shift_r
132   GENERIC MAP(DATA_WIDTH)
133   port map(

```

```

134     DIN    => Y,
135     SHIFT => shift_val_1,
136     DOUT   => Y_shift
137 );
138
139 -- X_new
140 ADD_X: ADDER
141 GENERIC MAP(DATA_WIDTH)
142 Port map (
143     A      => X,
144     B      => Y_shift,
145     RESULT => X_ADD
146 );
147
148 SUB_X: SUBT
149 GENERIC MAP(DATA_WIDTH)
150 Port map (
151     A      => X,
152     B      => Y_shift,
153     RESULT => X_SUB
154 );
155
156 X_new <= X_SUB when Y_Comp = '0' else X_ADD;
157
158 -- Y_new
159 ADD_Y: ADDER
160 GENERIC MAP(DATA_WIDTH)
161 Port map (
162     A      => Y,
163     B      => X_shift,
164     RESULT => Y_ADD
165 );
166
167 SUB_Y: SUBT
168 GENERIC MAP(DATA_WIDTH)
169 Port map (
170     A      => Y,
171     B      => X_shift,
172     RESULT => Y_SUB
173 );
174
175 Y_new <= Y_SUB when Y_Comp = '0' else Y_ADD;
176
177 -- Multiply with 1/K 1 / 0.828159 in Q8.8 = "0000 0001 0011 0101"
178 K <= std_logic_vector(to_signed(19776, 16));
179
180 Sqrt_MUL: Multiplier
181 PORT MAP( X, K, Sqrt);
182
183 -- Shift Sqrt
184 Shift_right_Sqrt: Shift_r

```

```

185  GENERIC MAP(DATA_WIDTH)
186  port map(
187      DIN    => Sqrt,
188      SHIFT => shift_val_2,
189      DOUT   => Sqrt_shift_right
190  );
191
192  Shift_left_Sqrt: Shift_1
193  GENERIC MAP(DATA_WIDTH)
194  port map(
195      DIN    => Sqrt,
196      SHIFT => shift_val_2,
197      DOUT   => Sqrt_shift_left
198  );
199
200  Sqrt_shift <= Sqrt_shift_left when Sc_lt = '0' else Sqrt_shift_right;
201
202  -- MUX 2-to-1
203  Sqrt_src <= Sqrt_shift when Sqrt_sel = '1' else "0000000000000000";
204
205  -- reg SQRT
206  RegSQRT : regn
207  Generic MAP (DATA_WIDTH)
208  PORT map (RST, CLK, Sqrt_ld, Sqrt_src, Sqrt_o);
209
210 end arc;

```

Listing 2: Nội dung tệp mô tả đường dữ liệu datapath.vhd

4.8 Mã nguồn VHDL Khối Controller

Dưới đây là nội dung toàn vẹn của tệp `controller.vhd`. Khối này triển khai Máy Trạng thái Hữu hạn (FSM) đóng vai trò là não bộ điều khiển toàn bộ các tín hiệu chọn, tín hiệu cho phép chốt và đồng bộ hóa hoạt động của đường dữ liệu theo thời gian.

```

1  LIBRARY IEEE;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY controller IS
6  PORT(
7      RST, CLK      : IN STD_LOGIC;
8      Start_i       : IN STD_LOGIC;
9
10     a_comp         : IN STD_LOGIC;
11     a_lt_0         : IN STD_LOGIC;
12     A_gt           : IN STD_LOGIC;
13     A_lt           : IN STD_LOGIC;
14     i_comp         : IN STD_LOGIC;
15
16     A_ld            : OUT STD_LOGIC;

```

```

17
18     Sc_sel      : OUT STD_LOGIC;
19     Sc_ld       : OUT STD_LOGIC;
20     Sc_new_sel  : OUT STD_LOGIC;
21
22     i_sel       : OUT STD_LOGIC;
23     i_ld        : OUT STD_LOGIC;
24
25     A_sel       : OUT STD_LOGIC;
26     X_sel       : OUT STD_LOGIC;
27     Y_sel       : OUT STD_LOGIC;
28
29     X_ld        : OUT STD_LOGIC;
30     Y_ld        : OUT STD_LOGIC;
31
32     A_shift_sel : OUT STD_LOGIC;
33
34     Sqrt_ld     : OUT STD_LOGIC;
35     Sqrt_sel    : OUT STD_LOGIC;
36
37     Done_o      : OUT STD_LOGIC;
38     err         : OUT STD_LOGIC
39 );
40 END controller;
41
42 ARCHITECTURE bev OF controller IS
43
44     TYPE State_type IS (
45         S0, S1, S2, S3, S4, S5, S6, S7,
46         S8, S9, S10, S11, S12, S13, S14, S15, S16
47     );
48     SIGNAL State : State_type;
49
50 BEGIN
51
52
53     PROCESS(RST, CLK)
54     BEGIN
55
56         IF RST = '1' THEN
57             State <= S0;
58         ELSIF rising_edge(CLK) THEN
59
60             CASE State IS
61
62
63                 WHEN S0 =>
64                     State <= S1;
65
66                 WHEN S1 =>
67

```

```

68         IF Start_i = '1' THEN
69
70             IF a_comp = '1' THEN
71                 State <= S2;
72             ELSIF a_lt_0 = '1' THEN
73                 State <= S3;
74             ELSE
75                 State <= S4;
76             END IF;
77
78         ELSE
79             State <= S1;
80         END IF;
81
82
83         -- input = 0
84         WHEN S2 =>
85             State <= S16;
86
87         -- input < 0
88         WHEN S3 =>
89             State <= S16;
90
91         -- init scale
92         WHEN S4 =>
93             State <= S5;
94         -- load A
95         WHEN S5 =>
96             State <= S6;
97
98         -- normalize
99         WHEN S6 =>
100
101             IF A_lt = '1' THEN
102                 State <= S7;
103             ELSIF A_gt = '1' THEN
104                 State <= S8;
105             ELSE
106                 State <= S9;
107             END IF;
108
109
110         -- shift left A
111         WHEN S7 =>
112             State <= S6;
113
114         -- shift right A
115         WHEN S8 =>
116             State <= S6;
117
118         -- init X,Y

```



```

119     WHEN S9 =>
120         State <= S10;
121
122         -- init i
123     WHEN S10 =>
124         State <= S11;
125
126         -- iteration loop
127     WHEN S11 =>
128
129         IF i_comp = '1' THEN
130             State <= S15;
131         ELSE
132             State <= S12;
133         END IF;
134
135
136     WHEN S12 =>
137         State <= S13;
138
139     WHEN S13 =>
140         State <= S14;
141
142     WHEN S14 =>
143         State <= S11;
144
145     WHEN S15 =>
146         State <= S16;
147
148     WHEN S16 =>
149         State <= S0;
150     WHEN OTHERS =>
151         State <= S0;
152 END CASE;
153
154 END IF;
155
156 END PROCESS;
157
158
159 PROCESS(State)
160 BEGIN
161
162
163
164     A_ld <= '0';
165     Sc_sel <= '0';
166     Sc_ld <= '0';
167     Sc_new_sel <= '0';
168
169     i_sel <= '0';

```

```

170     i_ld <= '0';
171
172     A_sel <= '0';
173     X_sel <= '0';
174     Y_sel <= '0';
175
176     X_ld <= '0';
177     Y_ld <= '0';
178
179     A_shift_sel <= '0';
180
181     Sqrt_ld <= '0';
182     Sqrt_sel <= '0';
183
184     Done_o <= '0';
185     err <= '0';
186
187
188     CASE State IS
189
190
191         WHEN S2 =>
192             -- sqrt(0)=0
193             Sqrt_sel <= '0';
194             Sqrt_ld <= '1';
195
196
197         WHEN S3 =>
198             err <= '1';
199
200             -- init scale = 0
201         WHEN S4 =>
202             Sc_sel <= '0';
203             Sc_ld <= '1';
204
205
206             -- load A
207         WHEN S5 =>
208             A_sel <= '0';
209             A_ld <= '1';
210
211
212             -- A = A << 2
213             -- scale--
214         WHEN S7 =>
215             A_shift_sel <= '0';
216             Sc_sel <= '1';
217             Sc_ld <= '1';
218             Sc_new_sel <= '0';
219             A_sel <= '1';
220             A_ld <= '1';

```

```

221
222      -- A = A >> 2
223      -- scale++
224      WHEN S8 =>
225          A_shift_sel <= '1';
226          Sc_sel <= '1';
227          Sc_ld <= '1';
228          Sc_new_sel <= '1';
229          A_sel <= '1';
230          A_ld <= '1';
231
232      -- X = A + 0.25
233      -- Y = A - 0.25
234      WHEN S9 =>
235          X_sel <= '0';
236          X_ld <= '1';
237          Y_sel <= '0';
238          Y_ld <= '1';
239
240
241      -- i = 1
242      WHEN S10 =>
243          i_sel <= '0';
244          i_ld <= '1';
245
246      WHEN S13 =>
247          X_sel <= '1';
248          X_ld <= '1';
249          Y_sel <= '1';
250          Y_ld <= '1';
251
252
253      -- i++
254      WHEN S14 =>
255          i_sel <= '1';
256          i_ld <= '1';
257
258      -- update sqrt
259      WHEN S15 =>
260          Sqrt_sel <= '1';
261          Sqrt_ld <= '1';
262
263      WHEN S16 =>
264          Done_o <= '1';
265      WHEN OTHERS =>
266          NULL;
267      END CASE;
268
269  END PROCESS;
270
271  END bev;

```

4.9 Mã nguồn VHDL Khối Testbench (Mô phỏng)

Dưới đây là nội dung tệp SQRT_tb.vhd, dùng để tạo môi trường mô phỏng (Testbench) kiểm tra hoạt động của toàn bộ hệ thống tính căn bậc hai với các trường hợp dữ liệu đầu vào (*Test cases*) khác nhau.

```

1  LIBRARY IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use work.Mylib.all;
5
6  ENTITY SQRT_tb IS
7  END SQRT_tb;
8
9  ARCHITECTURE bev OF SQRT_tb IS
10     Constant DATA_WIDTH : integer := 16;
11
12     SIGNAL RST, CLK : STD_LOGIC := '0';
13     SIGNAL Start_i   : STD_LOGIC;
14     SIGNAL a_i       : STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
15     SIGNAL SQRT_o    : STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
16     SIGNAL Done_o    : STD_LOGIC;
17 BEGIN
18
19     -- Khi to Unit Under Test (UUT)
20     UUT: SQRT
21     GENERIC MAP (DATA_WIDTH)
22     PORT MAP(
23         CLK, RST, Start_i,
24         a_i,
25         Done_o,
26         Sqrt_o
27     );
28
29     -- To xung nhp Clock (Chu k 20ns)
30     CLK <= not CLK after 10 ns;
31
32     -- Tin trnh to kch thch (Stimulus Process)
33     Stimulus: PROCESS
34     BEGIN
35         -- case 0: Khi to h thng
36         START_i <= '0';
37         RST <= '1';
38         wait for 20 ns;
39
40         RST <= '0';
41         wait for 20 ns;

```

```

42
43      -- case 1: a_i = 0
44      a_i <= "0000000000000000";
45      Start_i <= '1';
46      wait until Done_o = '1';
47      Start_i <= '0';
48      wait for 100 ns;
49
50      -- case 2: a_i = 0.25
51      a_i <= "0000000001000000";
52      Start_i <= '1';
53      wait until Done_o = '1';
54      Start_i <= '0';
55      wait for 100 ns;
56
57      -- case 3: a_i = 0.5
58      a_i <= "0000000010000000";
59      Start_i <= '1';
60      wait until Done_o = '1';
61      Start_i <= '0';
62      wait for 100 ns;
63
64      -- case 4: a_i = 1
65      a_i <= "0000000100000000";
66      Start_i <= '1';
67      wait until Done_o = '1';
68      Start_i <= '0';
69      wait for 100 ns;
70
71      -- case 5: a_i = 4
72      a_i <= "0000010000000000";
73      Start_i <= '1';
74      wait until Done_o = '1';
75      Start_i <= '0';
76      wait for 100 ns;
77
78      END PROCESS;
79  END BEV;

```

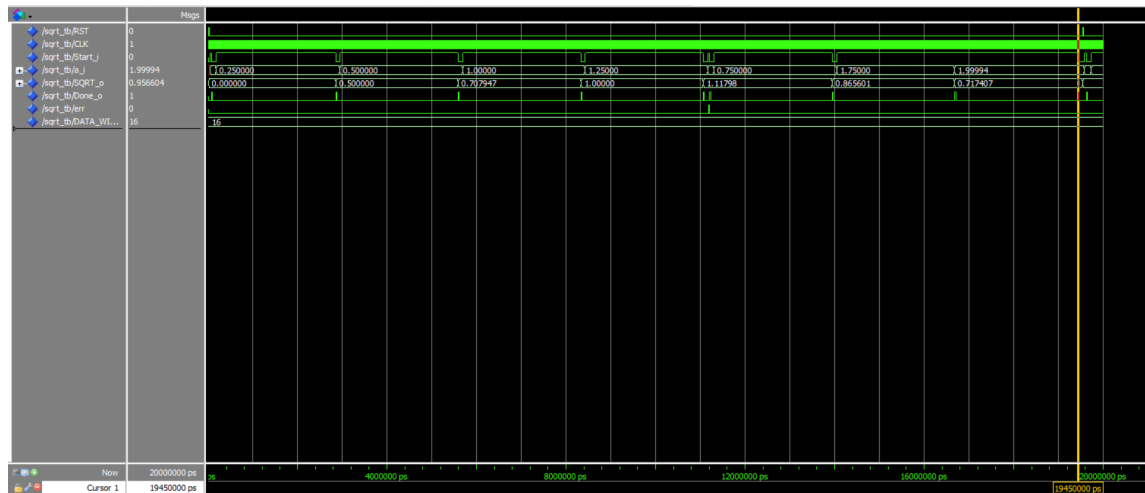
Listing 4: Nội dung tệp kiểm tra mô phỏng SQRT_tb.vhd

5 Mô phỏng/thực thi và đánh giá

Mạch được mô phỏng chức năng trên phần mềm **ModelSim** để kiểm thử, sau đó được tổng hợp trên **Vivado Design Suite**. Kết quả mô phỏng như sau:

5.1 Mô phỏng trên ModelSim

Kết quả mô phỏng



Hình 9: Kết quả mô phỏng bằng phần mềm ModelSim

Nhận xét

- Ta thấy mạch hoạt động đúng chức năng mong muốn, sai số như sau:

`sqrt(0.000000):` CORDIC = 0.000000000, math = 0.000000000, sai số = 0.00e+00

`sqrt(0.250000):` CORDIC = 0.500000, math = 0.500000, sai số = 0.00e+00

`sqrt(0.500000):` CORDIC = 0.707947, math = 0.707106, sai số = 8.41e-03

`sqrt(0.750000):` CORDIC = 0.865601, math = 0.866025, sai số = 0.01e+00

`sqrt(1.000000):` CORDIC = 1.000000, math = 1.000000, sai số = 0.00e+00

`sqrt(1.250000):` CORDIC = 1.11798, math = 1.11803, sai số = 5e-5

`sqrt(-0.250000):` CORDIC = err, math = err, sai số = 0.00e+00

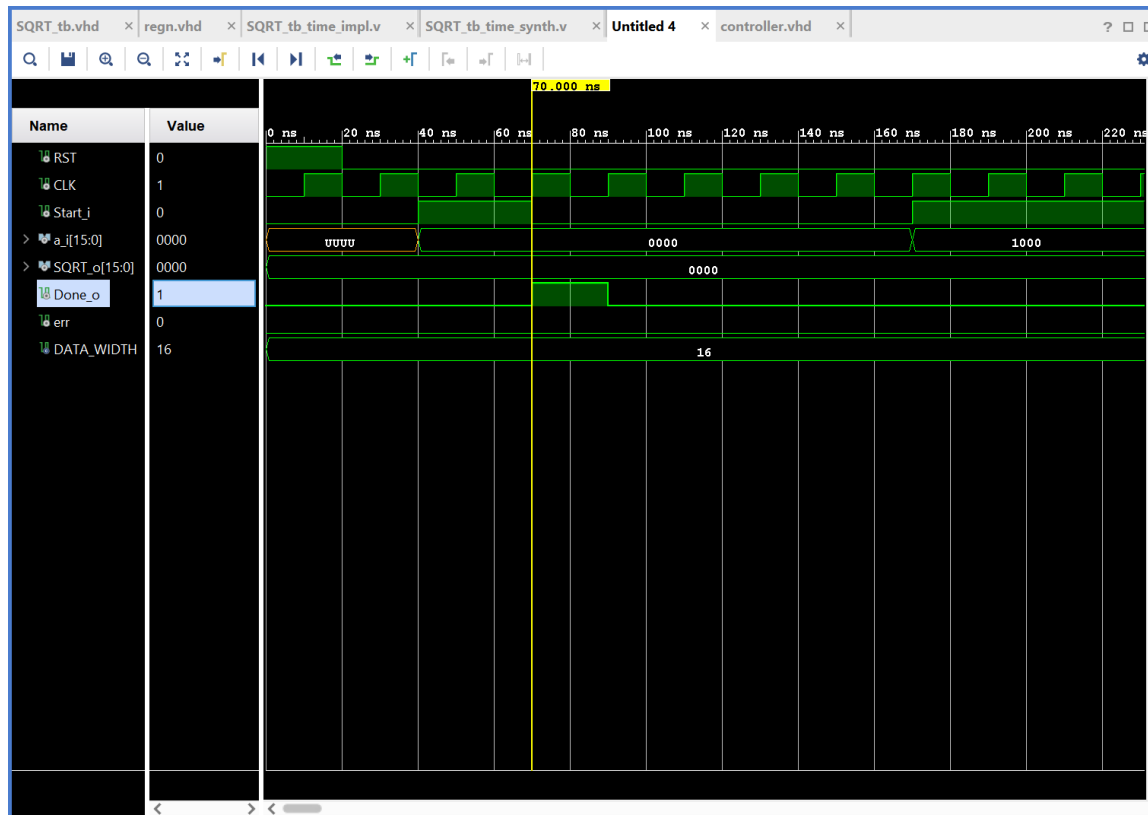
6 Triển khai trên FPGA

(Chỉ ra công cụ và bo mạch FPGA được sử dụng. Kết quả tổng hợp (synthesis report): số LUTs, FFs, DSPs, BRAMs, tần số hoạt động tối đa; kết quả Post-implementation simulation. Thực nghiệm trên phần cứng (đo đạc bằng ILA/VIO, triển khai SoC,...))

Mạch được tổng hợp trên phần mềm **Vivado**.

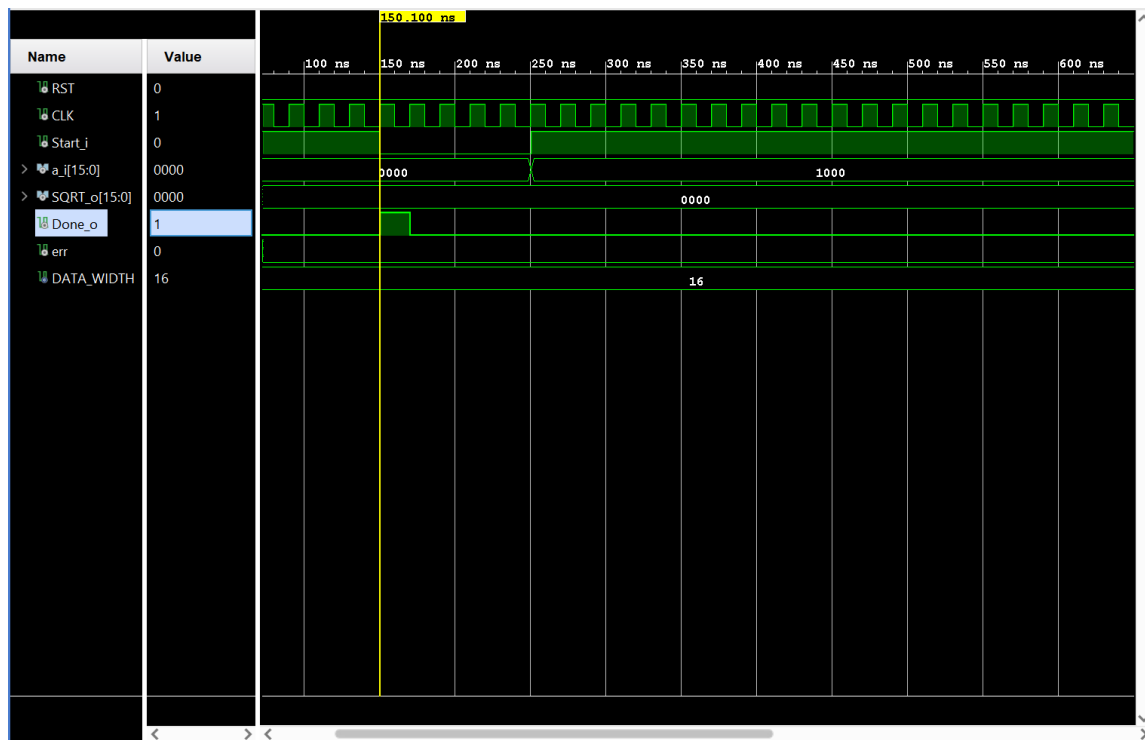
6.1 Tổng hợp kết quả trên Vivado

Behavioral Simulation



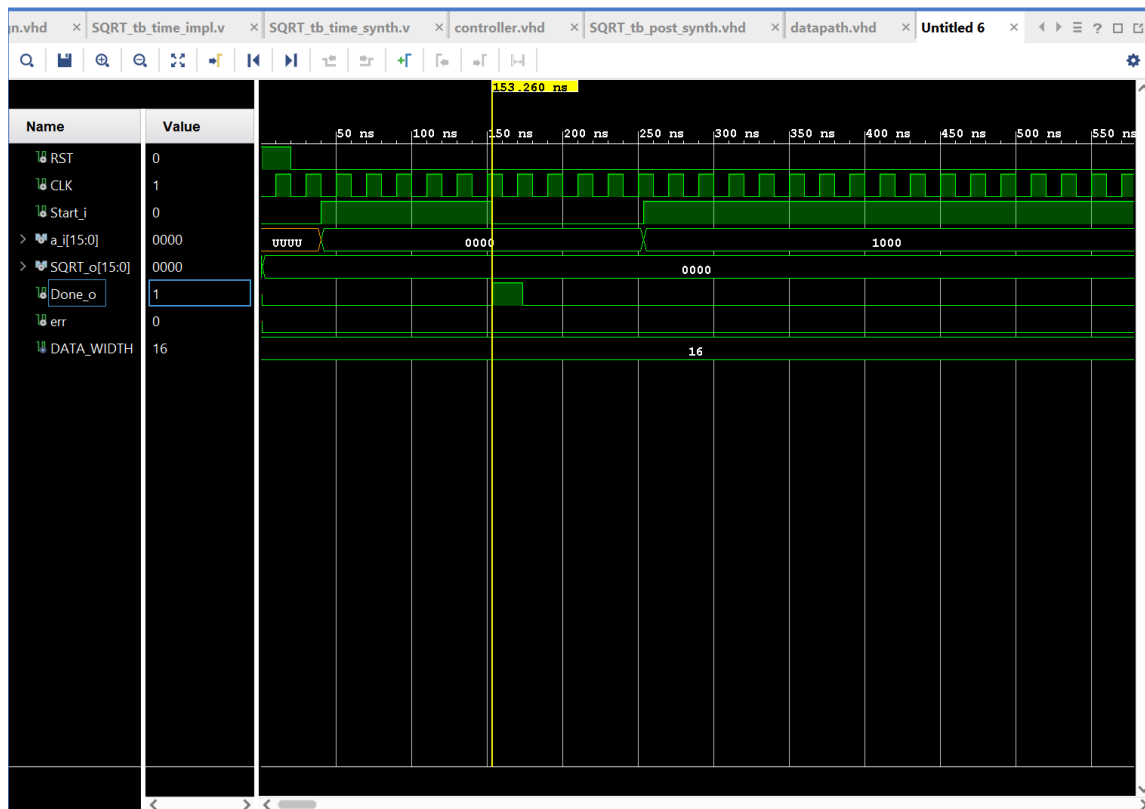
Hình 10: Behavioral Simulation

Post-Synthesis Functional Simulation



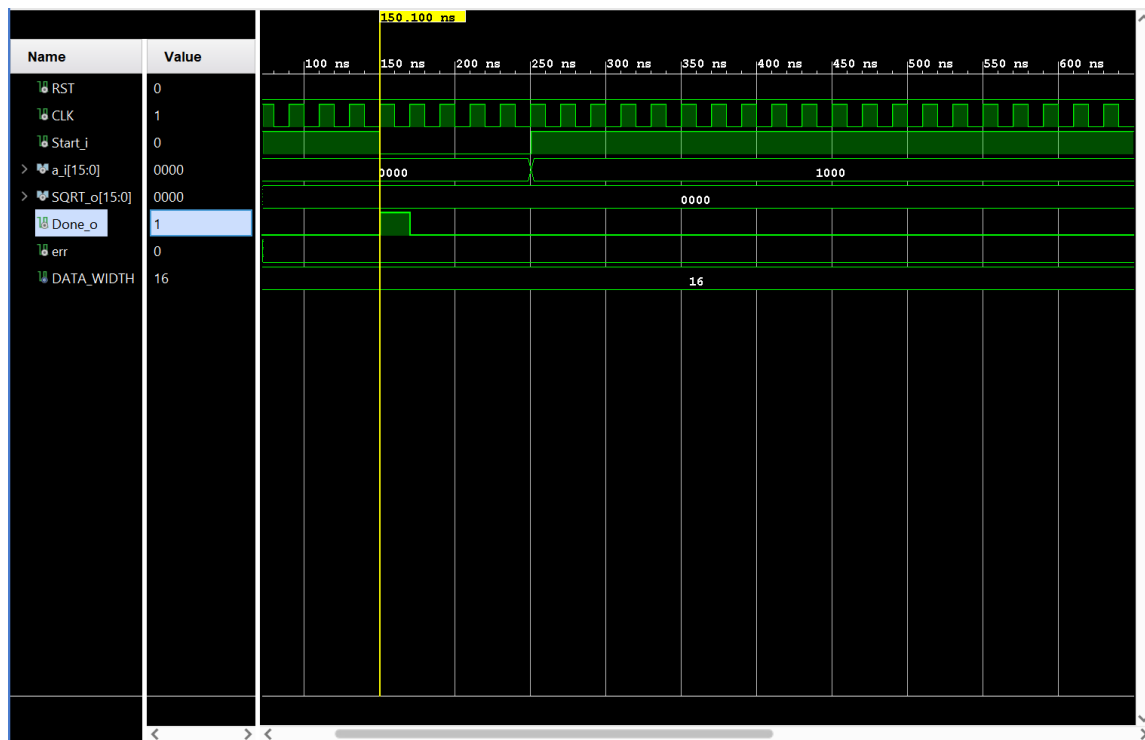
Hình 11: Post-Synthesis Functional Simulation

Post-Synthesis Timing Simulation



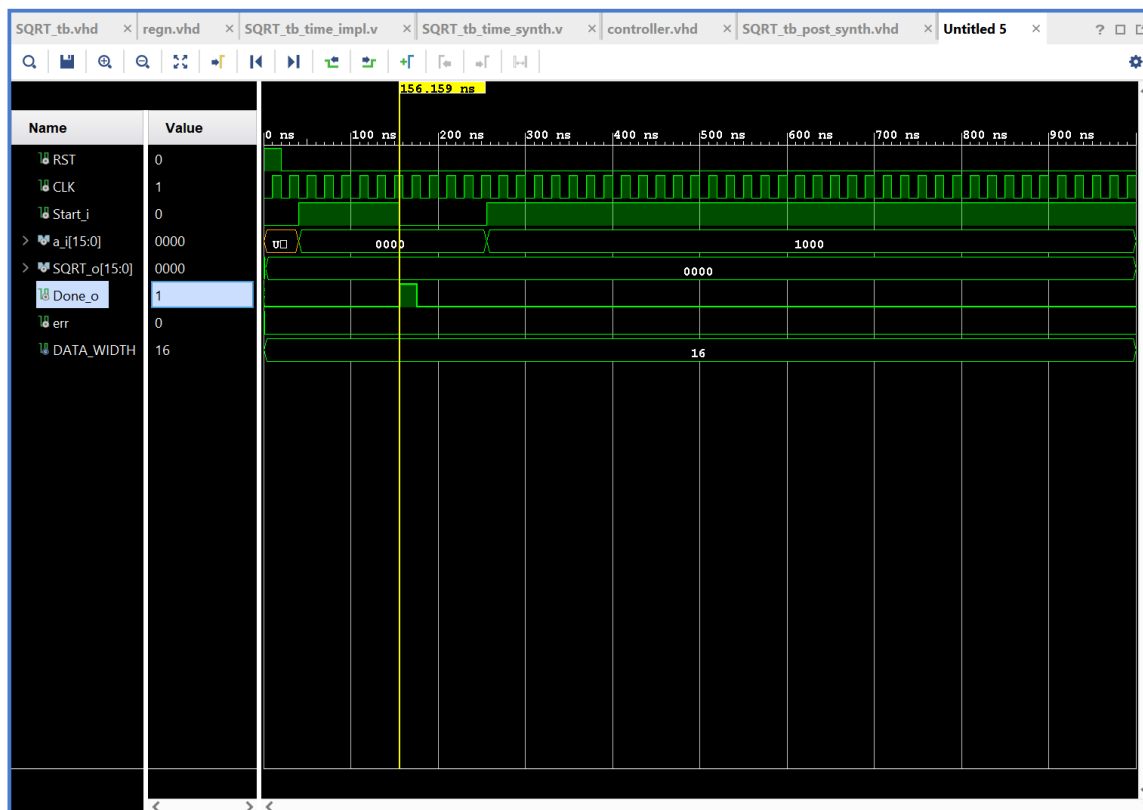
Hình 12: Post-Synthesis Timing Simulation

Post-Implementation Functional Simulation



Hình 13: Post-Implementation Functional Simulation

Post-Implementation Timing Simulation



Hình 14: Post-Implementation Timing Simulation

Phân tích kết quả Timing Simulation

Dựa trên kết quả mô phỏng của tín hiệu Done_o (chuyển lên mức 1), ta ghi nhận các mốc thời gian hoàn tất khác nhau qua 3 giai đoạn:

- Mô phỏng Hành vi (Behavioral Simulation): 70.000 ns
- Mô phỏng Sau tổng hợp (Post-Synthesis Timing Simulation): 153.260 ns
- Mô phỏng Sau thực thi (Post-Implementation Timing Simulation): 156.159 ns

7 Đánh giá sau Post-Implementation

7.1 Kết quả mô phỏng

Sau khi thực hiện post-implementation, mạch vẫn hoạt động đúng chức năng khi kiểm tra bằng *functional simulation*. Tuy nhiên, khi thực hiện *post-implementation timing simulation*, tại một số thời điểm tín hiệu đầu ra xuất hiện nhiễu ngắn trước khi ổn định tại giá trị cuối cùng.

7.2 Phân tích Timing

- WNS (Worst Negative Slack) = 2.645 ns: tín hiệu đến đích sớm hơn 2.645 ns so với yêu cầu của chu kỳ xung nhịp (thoả mãn điều kiện setup time).
- TNS (Total Negative Slack) = 0 ns và Failing Endpoints = 0: không có bất kỳ đường truyền nào trong mạch vi phạm điều kiện timing.

Đánh giá: Mạch hoạt động ổn định và an toàn tại mức xung nhịp thiết lập là 10 ns (tương đương 100 MHz).

7.3 Tính toán tần số hoạt động tối đa

Chu kỳ xung nhịp mong muốn:

$$T_{clk} = 10 \text{ ns}$$

Với thời gian dư (Worst-case slack):

$$WNS = 2.645 \text{ ns}$$

Chu kỳ tối thiểu mà mạch yêu cầu:

$$T_{min} = T_{clk} - WNS = 10 - 2.645 = 7.355 \text{ ns}$$

Tần số hoạt động tối đa:

$$F_{max} = \frac{1}{T_{min}} = \frac{1}{7.355 \times 10^{-9}} \approx 135.96 \times 10^6 \text{ Hz}$$

$$F_{max} \approx 135.96 \text{ MHz}$$

7.4 Kết luận

Chu kỳ thực thi tối thiểu của mạch là 7.355 ns. Do đó, tần số hoạt động tối đa của mạch SQRT có thể đạt được một cách an toàn mà không vi phạm điều kiện timing là khoảng 135.96 MHz.

Nhận xét: Sau quá trình tổng hợp (synthesis), mạch vẫn đảm bảo đúng chức năng theo yêu cầu bài toán. Tuy nhiên, khi xét đến timing simulation, tín hiệu sẽ xuất hiện độ trễ nhất định so với kết quả của functional simulation, đây là hiện tượng bình thường do ảnh hưởng của các phần tử vật lý trong phần cứng. Sự thay đổi thời gian:

$$70.000 \text{ ns} \rightarrow 153.260 \text{ ns} \rightarrow 156.159 \text{ ns}$$

là minh chứng cho quá trình chuyển đổi từ mô hình thuật toán lý tưởng sang một mạch điện vật lý thực tế.

Việc thực hiện đầy đủ cả 3 bước mô phỏng là cần thiết nhằm:

- Đảm bảo thiết kế hoạt động đúng chức năng logic.
- Phát hiện và kiểm tra các vi phạm timing (*timing violations*).
- Đánh giá khả năng hoạt động ổn định của hệ thống khi triển khai trên FPGA thực tế.

7.5 So sánh Functional Simulation giữa các giai đoạn

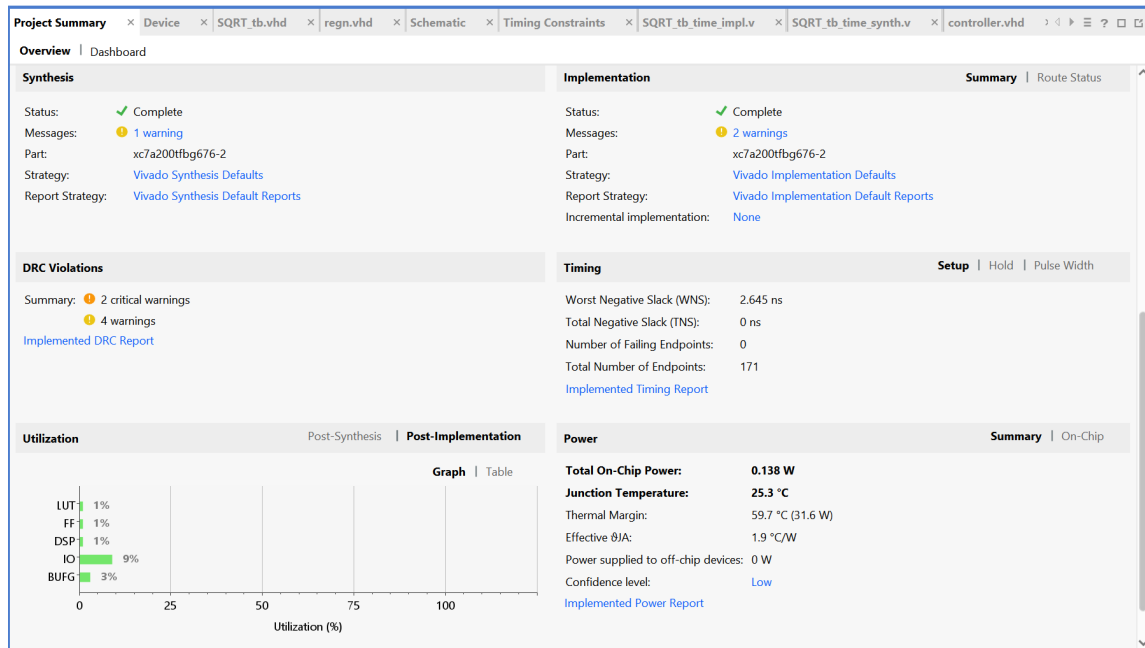
Khi chạy **Functional Simulation** ở cả hai giai đoạn:

- **Post-Synthesis Functional Simulation**
- **Post-Implementation Functional Simulation**

kết quả thu được hoàn toàn giống với **Behavioral Simulation**. Nguyên nhân gồm:

- **Bản chất Zero-Delay**
 - Functional Simulation chỉ nhằm kiểm tra tính đúng đắn của logic như:
 - * Thuật toán
 - * Phương trình Boolean
 - * Máy trạng thái (FSM)
 - Trình mô phỏng bỏ qua toàn bộ thông tin trễ vật lý của mạch.
 - Mọi sự chuyển trạng thái được xem là lý tưởng và có độ trễ bằng 0 ns.
- **Tính bảo toàn logic (Logical Equivalence)**
 - Quá trình:
 - * Synthesis
 - * Implementationchỉ thay đổi cấu trúc vật lý của mạch nhưng không được phép làm thay đổi chức năng logic ban đầu của thiết kế RTL.
 - Vì vậy, nếu không xét đến yếu tố thời gian, đầu ra ở tất cả các giai đoạn phải giống nhau.
- **Khác nhau về Netlist nhưng giống nhau về hành vi**
 - Ở **Behavioral Simulation**, phần mềm mô phỏng trực tiếp từ mã VHDL/Verilog.
 - Ở **Post-Synthesis Functional** và **Post-Implementation Functional**, mô phỏng được thực hiện trên *Netlist*.
 - Tuy nhiên, do không xét đến:
 - * Gate Delay
 - * Routing Delaynên kết quả waveform vẫn đồng bộ chu kỳ (*cycle-accurate*) và hoàn toàn giống nhau.

7.6 Report Summary



Hình 15: Project summary

- **WNS (Worst Negative Slack) = 2.645 ns:** Đây là thời gian "dư dả" tại đường dẫn (path) có độ trễ tệ nhất trong mạch. Giá trị dương (+) chứng tỏ tín hiệu đã đến đích sớm hơn **2.645 ns** so với yêu cầu của chu kỳ xung nhịp (Setup time).
- **TNS (Total Negative Slack) = 0 ns & Failing Endpoints = 0:** Không có bất kỳ đường dẫn nào trong mạch bị vi phạm timing.

Đánh giá: Mạch sẽ hoạt động ổn định và an toàn tại mức xung nhịp đang thiết lập (**10 ns** tương đương **100 MHz**), không có nguy cơ bị lỗi sai logic do tín hiệu truyền chậm.

Tính toán Tần số hoạt động tối đa F_{max}

Chu kỳ xung nhịp mong muốn (T_{clk}) là **10 ns**. Với lượng thời gian dư ra ở trường hợp xấu nhất là $WNS = 2.645$ ns, ta tính chu kỳ xung nhịp tối thiểu (T_{min}) mà phần cứng này thực sự cần để hoàn thành xong một chu kỳ logic:

$$T_{min} = T_{clk} - WNS = 10 - 2.645 = 7.355 \text{ ns}$$

Dựa vào công thức tính tần số tối đa (F_{max}):

$$F_{max} = \frac{1}{T_{clk} - WNS}$$

Áp dụng số liệu thực tế của dự án:

$$F_{max} = \frac{1}{7.355 \text{ ns}} = \frac{1}{7.355 \times 10^{-9} \text{ s}}$$

$$F_{max} \approx 135.96 \times 10^6 \text{ Hz} \approx 135.96 \text{ MHz}$$

Kết luận: Chu kỳ thực thi tối thiểu của mạch là **7.355 ns**. Do đó, tần số hoạt động tối đa (F_{max}) mạch SQRT này có thể đạt được một cách an toàn mà không bị lỗi Timing là khoảng **135.96 MHz**. Tần số hiện tại được sử dụng (**100 MHz**) nằm trong ngưỡng rất an toàn.

7.7 So sánh tài nguyên sử dụng của Post Synthesis và Port Implementation

Dưới đây là hình ảnh chụp thống kê tài nguyên của hệ thống ở hai giai đoạn:

Utilization			
Post-Synthesis Post-Implementation			
Graph Table			
Resource	Estimation	Available	Utilization %
LUT	269	134600	0.20
FF	93	269200	0.03
DSP	1	740	0.14
IO	37	400	9.25
BUFG	1	32	3.13

Hình 16: Tài nguyên dự kiến sau quá trình Tổng hợp (Post-Synthesis)

Graph Table			
Resource	Utilization	Available	Utilization %
LUT	269	133800	0.20
FF	95	267600	0.04
DSP	1	740	0.14
IO	37	400	9.25
BUFG	1	32	3.13

Hình 17: Tài nguyên thực tế sau quá trình Triển khai (Post-Implementation)

Bảng so sánh chi tiết

Dựa trên hình ảnh báo cáo, ta có bảng tổng hợp chênh lệch sử dụng tài nguyên như sau:

Tài nguyên	Post-Synthesis	Post-Implementation	Chênh lệch
LUT	269	269	0
FF	93	95	+2
DSP	1	1	0
IO	37	37	0
BUFG	1	1	0

Bảng 3: So sánh số lượng tài nguyên sử dụng

Nhận xét và Đánh giá

Từ các dữ liệu trên, ta có thể rút ra một số nhận xét chi tiết sau:

1. **Mức độ sử dụng tổng thể:** Thiết kế sử dụng lượng logic rất nhỏ so với dung lượng phần cứng của chip FPGA (các thành phần đều sử dụng dưới 10% tài nguyên có sẵn). Chân IO có tỷ lệ sử dụng cao nhất (9.25%), trong khi các thành phần logic như LUT và FF đều ở mức rất thấp (xấp xỉ 0.2%).
2. **Độ chính xác của quá trình ước lượng (Estimation):** Báo cáo Post-Synthesis đã ước lượng tài nguyên rất sát với thực tế. Hầu hết các tài nguyên cốt lõi (LUT, DSP, IO, BUFG) hoàn toàn không có sự thay đổi về số lượng sử dụng khi chuyển sang bước Implementation.
3. **Sự thay đổi của thanh ghi (Flip-Flops - FF):** Số lượng FF sử dụng thực tế tăng nhẹ từ 93 (Post-Synthesis) lên 95 (Post-Implementation). Nguyên nhân thường do trong giai đoạn Implementation (cụ thể là Place and Route), phần mềm thực hiện các bước tối ưu hóa vật lý như nhân bản thanh ghi (register duplication) để đáp ứng các yêu cầu về thời gian (timing constraints) hoặc giải quyết các đường truyền tín hiệu quá dài.
4. **Sự khác biệt về tài nguyên khả dụng (Available):** Có sự điều chỉnh giảm nhẹ về tổng số tài nguyên khả dụng ở bước Implementation (LUT từ 134,600 xuống 133,800; FF từ 269,200 xuống 267,600). Điều này xảy ra bởi vì khi áp dụng các ràng buộc vật lý, một số lượng nhỏ tài nguyên bị phần mềm tự động vô hiệu hóa để đảm bảo nhường không gian cho cấu trúc định tuyến (routing) cố định của chip.

Kết luận: Thiết kế được triển khai trơn tru, không gặp phải hiện tượng phình to logic (logic bloat) nào. Kết quả sau Implementation hoàn toàn tương đồng và tối ưu đúng theo dự tính ban đầu.

8 Kết luận cả dự án

8.1 Những gì đã đạt được

Qua dự án, nhóm chúng em đã có thể áp dụng các kiến thức đã được học để tự thiết kế, mô phỏng một mạch số đơn giản, hoạt động với độ chính xác cao.

8.2 Lời kết

Qua dự án lần này, nhóm chúng em đã được trang bị rất nhiều kiến thức giá trị, từ thiết kế phần cứng cho tới mô hình hóa bằng ngôn ngữ mô tả phần cứng và mô phỏng, tổng hợp trên phần mềm. Tuy nhiên, đây là dự án đầu tiên của nhóm chúng em nên không thể tránh khỏi những sai sót. Đó sẽ là những bài học quý báu để chúng em tiếp tục học hỏi và nghiên cứu trong tương lai.

Qua đây, nhóm chúng em xin gửi lời cảm ơn chân thành tới thầy TS. Nguyễn Kiêm Hùng vì đã tận tình hướng dẫn và dẫn dắt chúng em trong suốt quá trình thực hiện dự án.